

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Информационная система центра полиграфических услуг

Студентка гр. 3343	_____	Лобова Е. И.
Студентка гр. 3343	_____	Гельман П. Е.
Студентка гр. 3342	_____	Смирнова Е. С.
Преподаватель	_____	Заславский М. М.

Санкт-Петербург

2026

ЗАДАНИЕ

НА ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ

Студентка Лобова Е. И.

Студентка Гельман П. Е.

Студентка Смирнова Е. С.

Группа 3343, 3342

Тема работы: Информационная система центра полиграфических услуг

Исходные данные:

Задача - сделать сервис для обеспечения работы центра полиграфии (печать, сканирование, ризография). Роли: клиенты, сотрудники, администраторы.

Содержание пояснительной записки:

«Содержание», «Введение», «Сценарии использования», «Модель данных», «Разработанное приложение», «Заключение», «Список использованных источников».

Предполагаемый объем пояснительной записки:

Не менее 50 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 21.05.2026

Дата защиты реферата: 21.05.2026

Студентка

Лобова Е. И.

Студентка

Гельман П. Е.

Студентка

Смирнова Е. С.

Преподаватель

Заславский М. М.

АННОТАЦИЯ

Разработанное веб-приложение представляет собой информационную систему для автоматизации работы центра полиграфических услуг, реализованную с использованием JavaScript, Vue.js и графовой базы данных Neo4j. Система поддерживает три роли пользователей (клиенты, сотрудники, администраторы) и предоставляет функционал для приёма и выполнения заказов на печать, сканирование и ризографию, обеспечивая удобный интерфейс, разграничение доступа и контроль статусов выполнения работ на всех этапах обслуживания.

SUMMARY

The developed web application is an information system for automating the operation of a print center, implemented using JavaScript, Vue.js, and the Neo4j graph database. The system supports three user roles (clients, employees, administrators) and provides functionality for accepting and fulfilling orders for printing, scanning, and risography, offering a user-friendly interface, access control, and status tracking of work execution at all stages of service.

СОДЕРЖАНИЕ

	Введение	7
1.	Сценарии использования	9
1.1.	Макет UI	9
1.2.	Сценарии использования для задачи	20
1.3.	Вывод о преобладающих операциях	29
2.	Модель данных	30
2.1.	Нереляционная модель данных	30
2.1.1	Графическое представление	30
2.1.2	Описание назначений коллекций, типов данных и сущностей	31
2.1.3	Оценка удельного объема информации, хранимой в модели	35
2.1.4	Запросы к модели, с помощью которых реализуются сценарии использования	38
2.2.	Реляционная модель данных	49
2.2.1	Графическое представление	49
2.2.2	Описание назначений коллекций, типов данных и сущностей	49
2.2.3	Оценка удельного объема информации, хранимой в модели	53
2.2.4	Запросы к модели, с помощью которых реализуются сценарии использования	55
2.3.	Сравнение моделей	62
2.3.1	Удельный объем информации	62
2.3.2	Запросы по отдельным юзкейсам	63
2.3.3	Вывод	63
3.	Разработанное приложение	65
3.1.	Описание приложения и использованные технологии	65
3.2.	Снимки экрана приложения	66
	Заключение	75
	Список использованных источников	77

Приложение А. Документация по сборке и разворачиванию приложения	78
Приложение В. Инструкция для пользователя	79

ВВЕДЕНИЕ

а) Актуальность решаемой проблемы

В современном мире малый и средний бизнес в сфере полиграфических услуг сталкивается с рядом организационных трудностей: ручная обработка заказов, отсутствие единой базы клиентов, сложность отслеживания статусов выполнения работ (печать, сканирование, ризография) и разрозненность данных между сотрудниками. Многие центры полиграфии продолжают использовать бумажные журналы, Excel-таблицы или универсальные CRM-системы, не адаптированные под специфику полиграфических услуг, а некоторые центры вообще не ведут учёт заказов и клиентов, полагаясь на память сотрудников или устные договорённости. Это приводит к потерям времени, ошибкам в заказах, недовольству клиентов, финансовым потерям и снижению эффективности работы. Таким образом, разработка специализированной информационной системы для центра полиграфических услуг является актуальной задачей, направленной на автоматизацию бизнес-процессов и повышение качества обслуживания.

б) Постановка задачи

Целью данной работы является разработка веб-приложения — информационной системы для обеспечения работы центра полиграфических услуг, поддерживающей три категории пользователей: клиенты, сотрудники и администраторы. Согласно исходным данным, в качестве базы данных обязательно используется Neo4j, остальные технологии выбраны самостоятельно. Система должна предоставлять следующий функционал:

- для клиентов — оформление заказов на печать, сканирование и ризографию, просмотр статусов и истории заказов;
- для сотрудников — приём и выполнение заказов, изменение статусов, ведение клиентской базы;

- для администраторов — управление пользователями, тарифами и услугами, импорт и экспорт данных, а также построение кастомизированной статистики по различным параметрам.

с) Предлагаемое решение

В качестве решения предлагается информационная система, построенная на архитектуре «клиент-сервер» с веб-интерфейсом. Фронтенд реализован на Vue, серверная часть на Node.js с Express предоставляет REST API. Бизнес-логика реализована в отдельном слое на стороне сервера (проверка ролей, управление статусами заказов, расчёт стоимости). Данные пользователей, заказов и услуг хранятся в Neo4j в виде графовой модели, доступ к базе осуществляется через отдельный слой с использованием официального JS-драйвера. Система реализует ролевую модель доступа: клиент видит только свои заказы, сотрудник — все заказы, администратор — полное управление, включая импорт/экспорт данных и построение кастомизированной статистики.

д) Качественные требования к решению

Разрабатываемая информационная система должна удовлетворять следующим качественным требованиям:

- Надёжность — корректная обработка ошибок ввода, сохранение целостности данных в Neo4j, отсутствие потери заказов.
- Удобство использования — интуитивно понятный интерфейс, минимум действий для создания заказа, наглядное отображение статусов.
- Безопасность — аутентификация и авторизация на основе JWT-токенов, хеширование паролей, разделение доступа по ролям.

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1. Макет UI

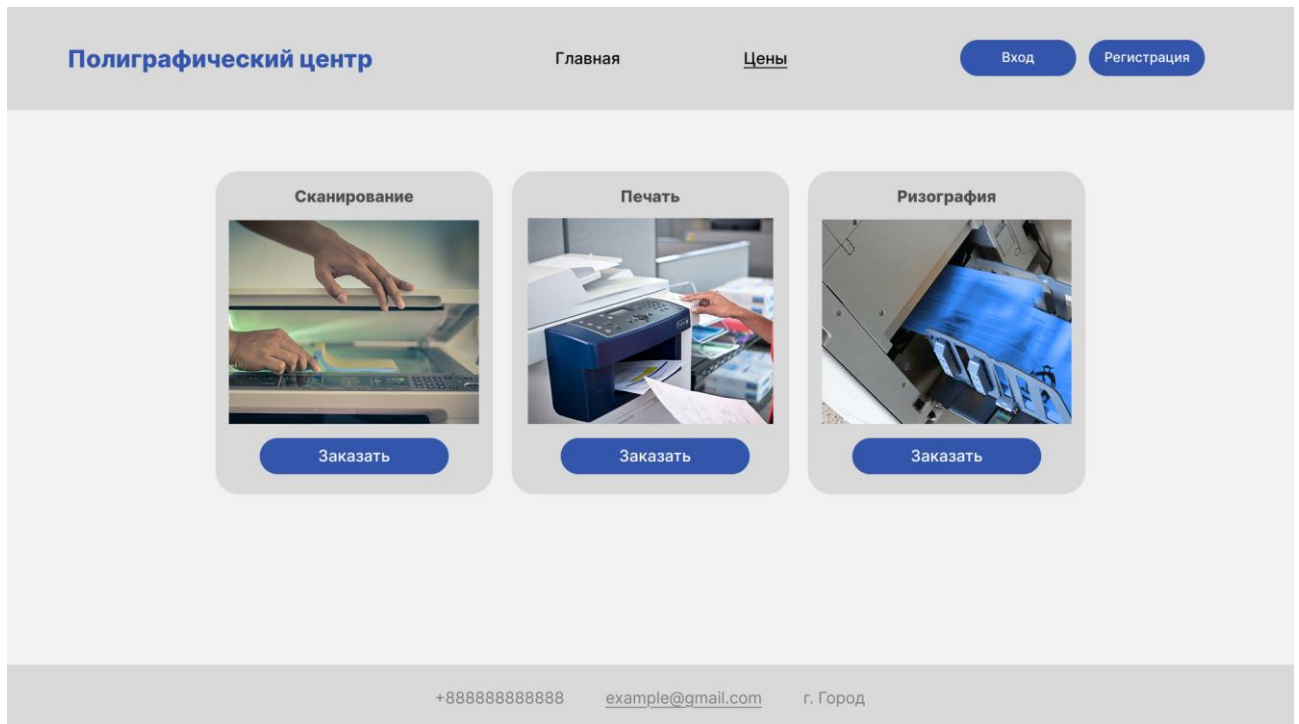


Рисунок 1 — Главная страница для клиента

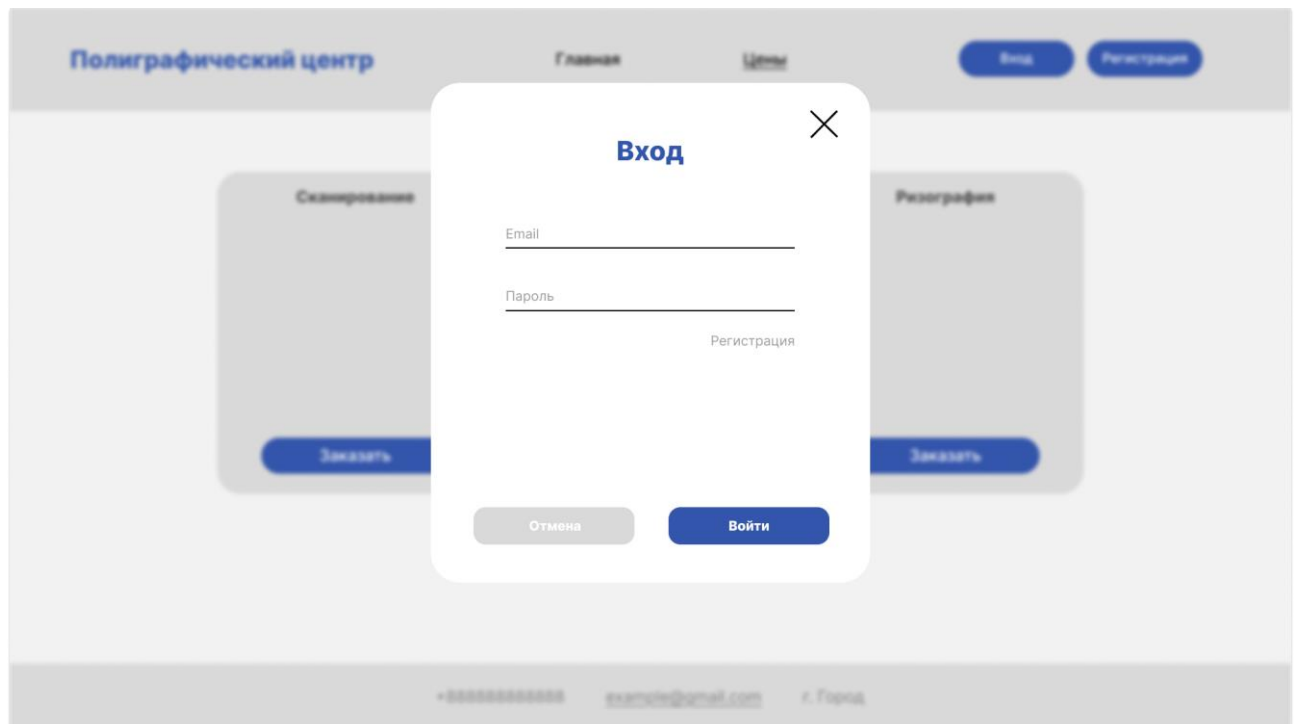


Рисунок 2 — Вход

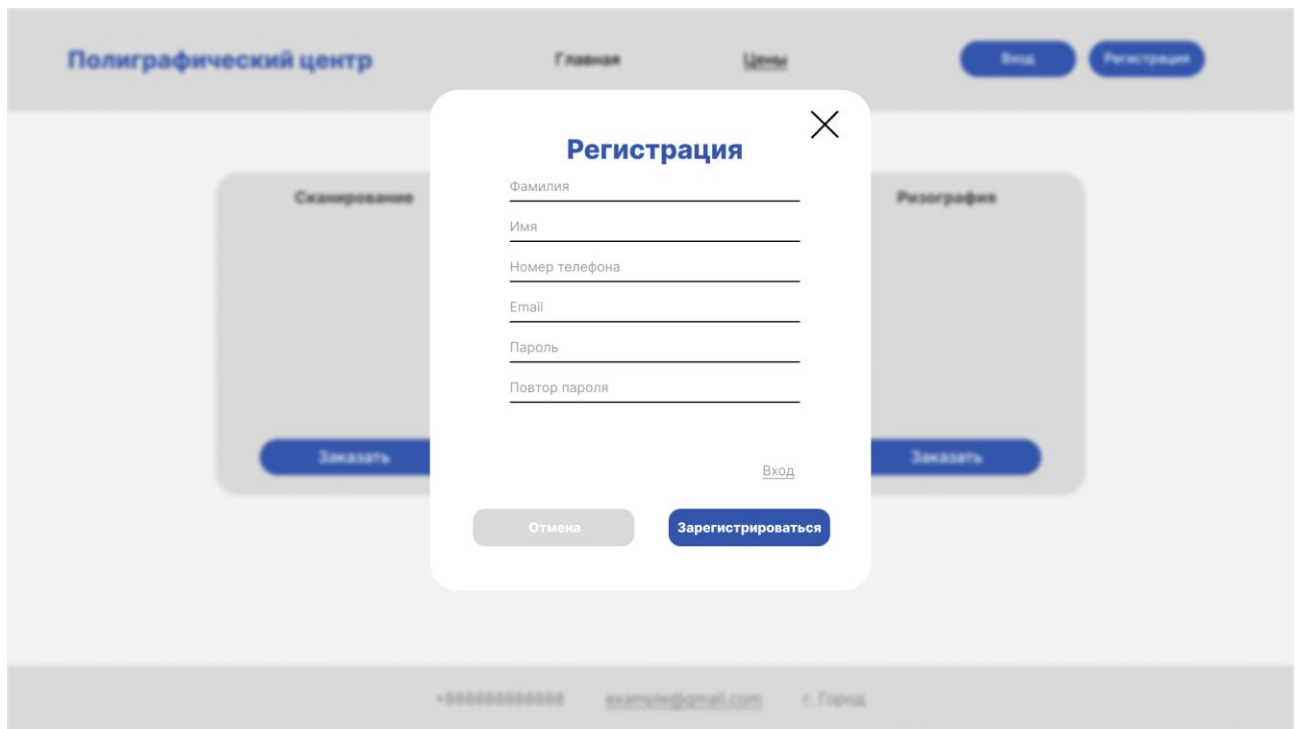


Рисунок 3 — Регистрация

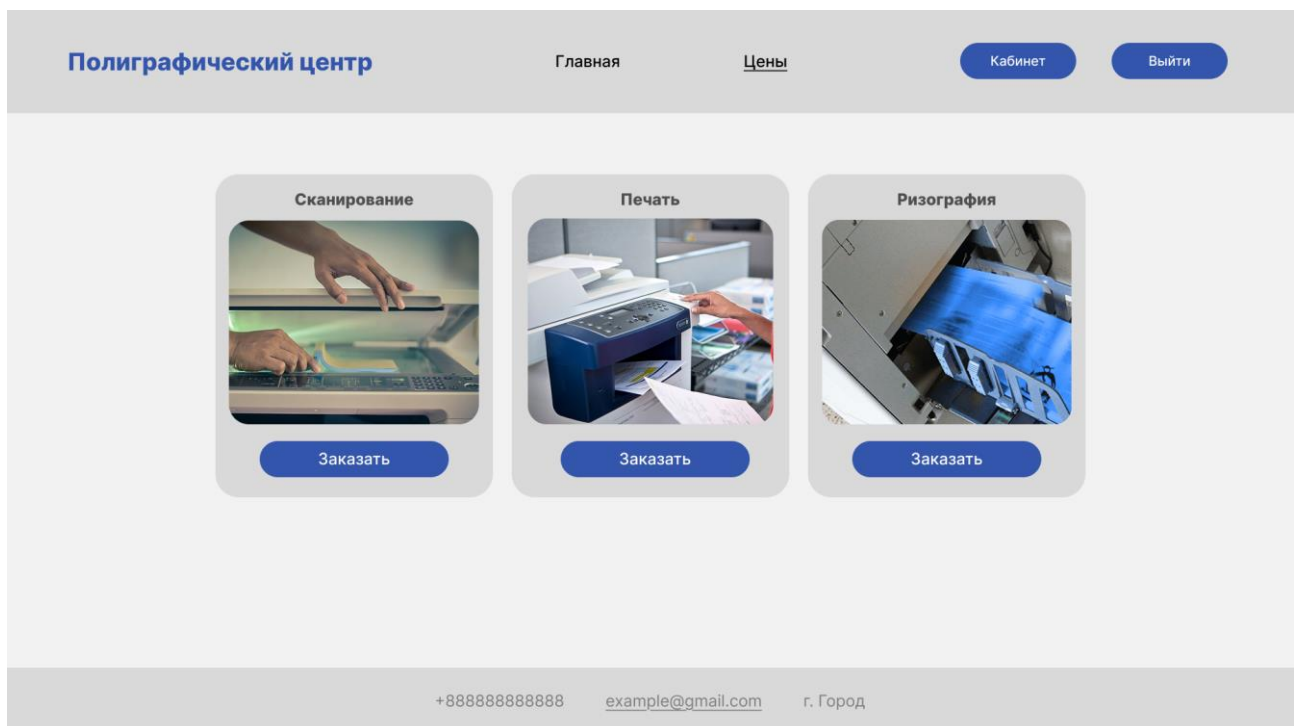


Рисунок 4 — Главная страница после авторизации

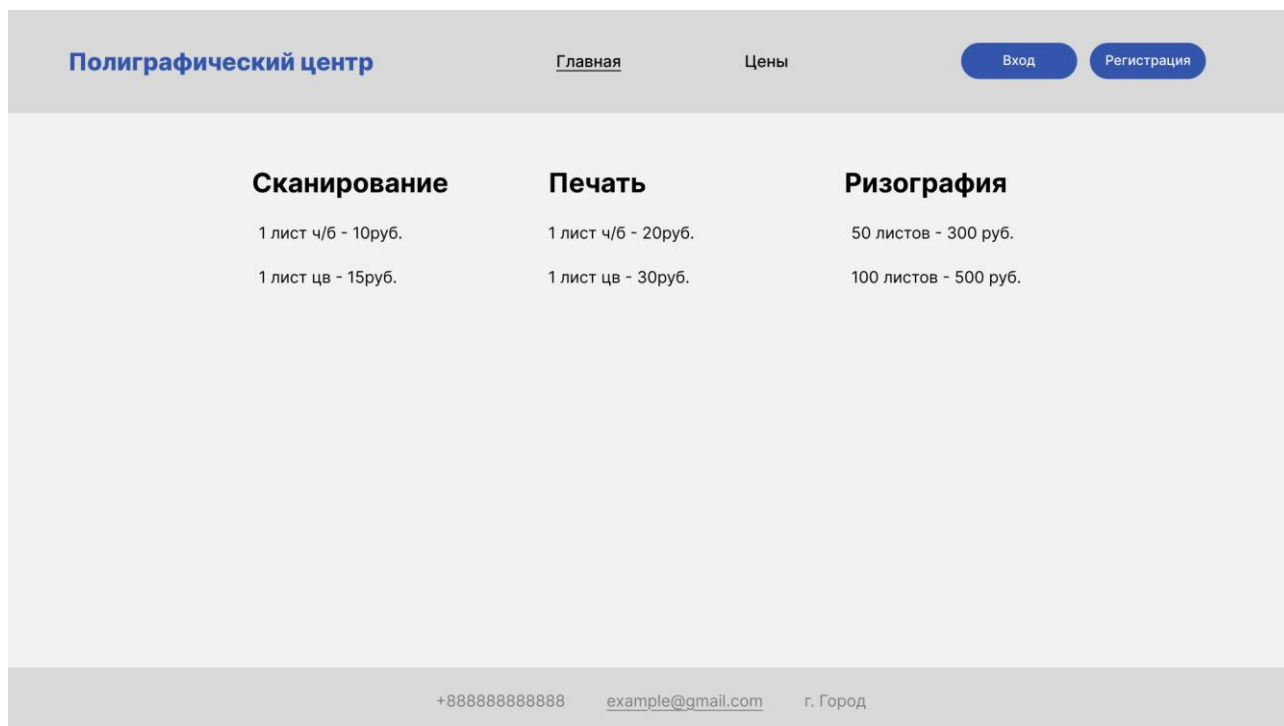


Рисунок 5 — Цены для клиента

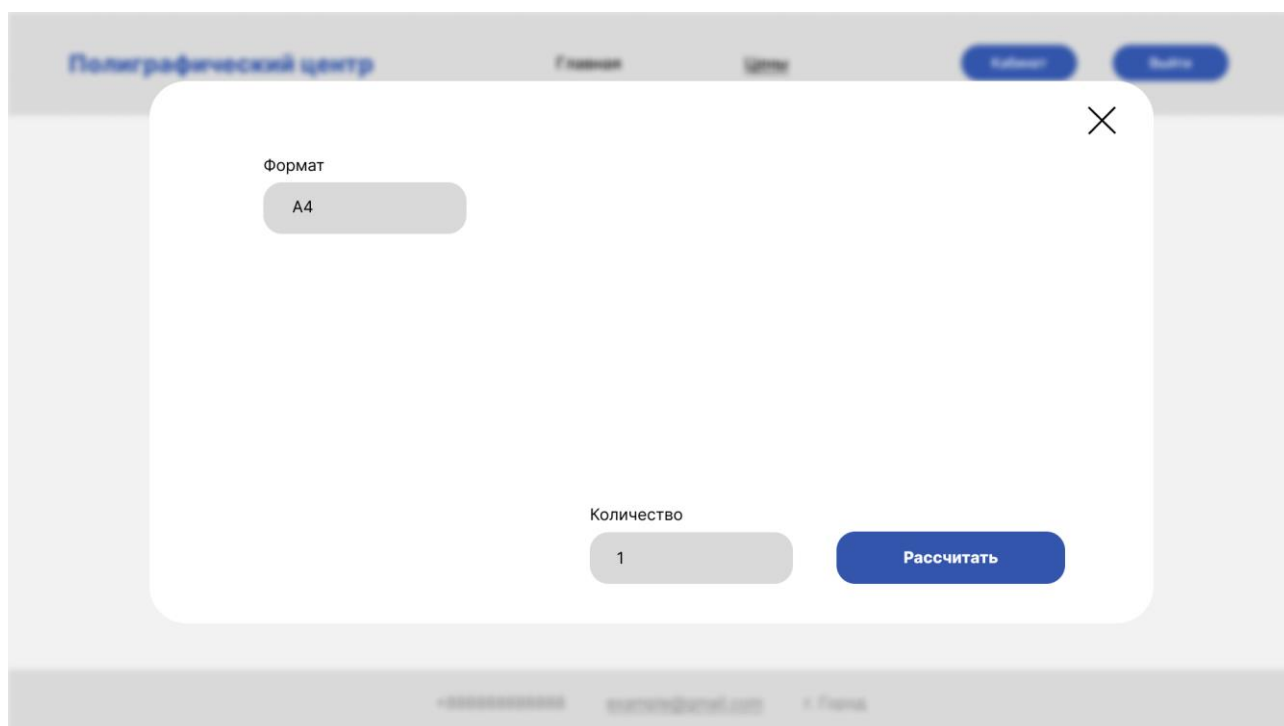


Рисунок 6 — Заказ сканирования

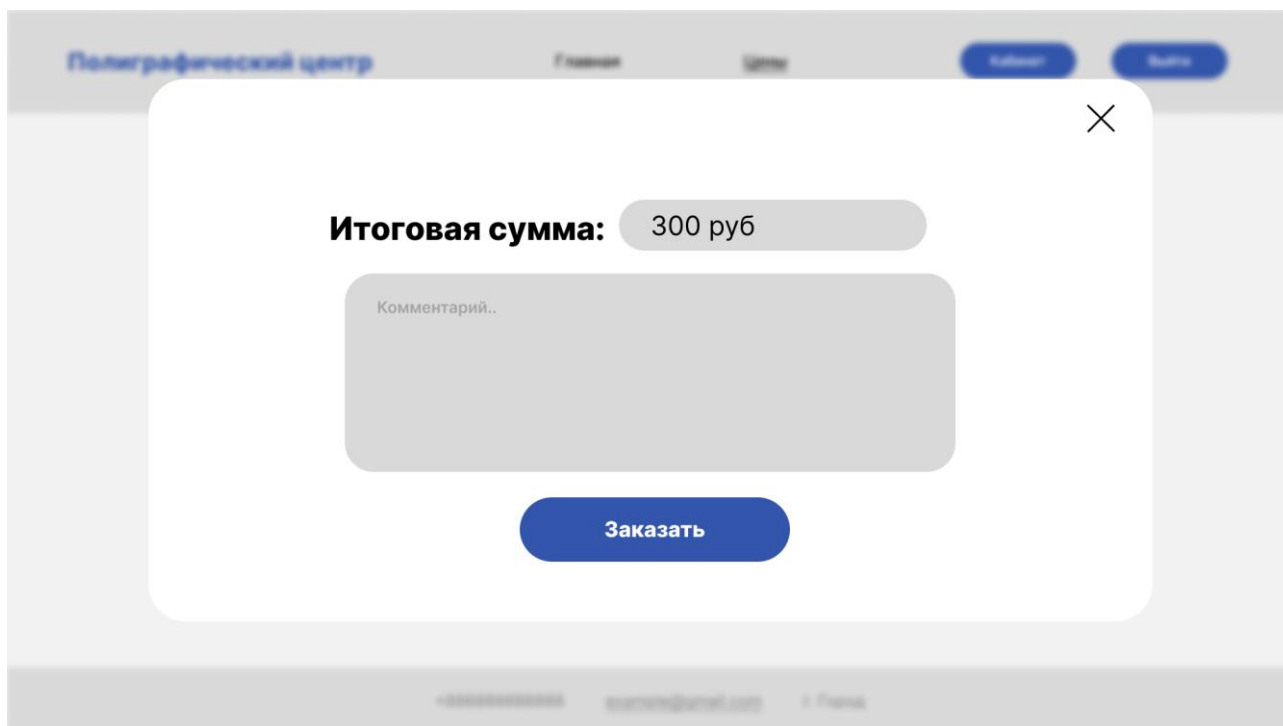


Рисунок 7 — Расчет стоимости заказа и поле комментариев

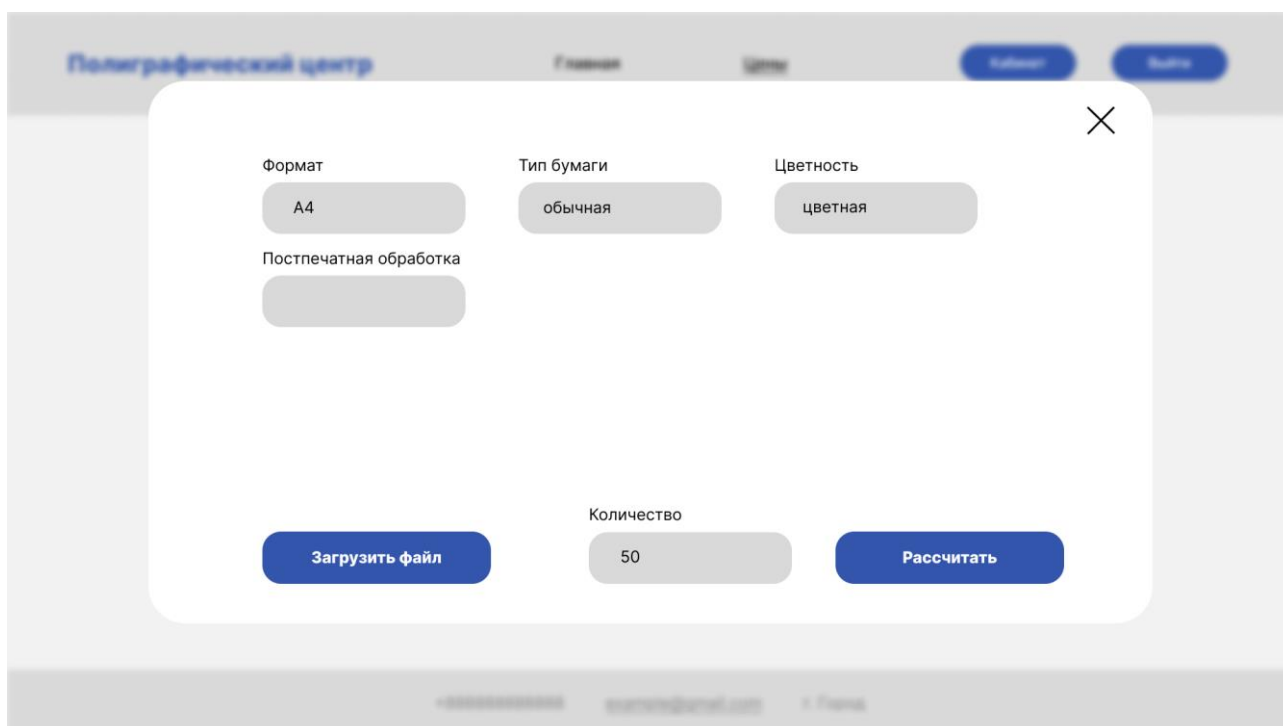


Рисунок 8 — Заказ печати/ризографии

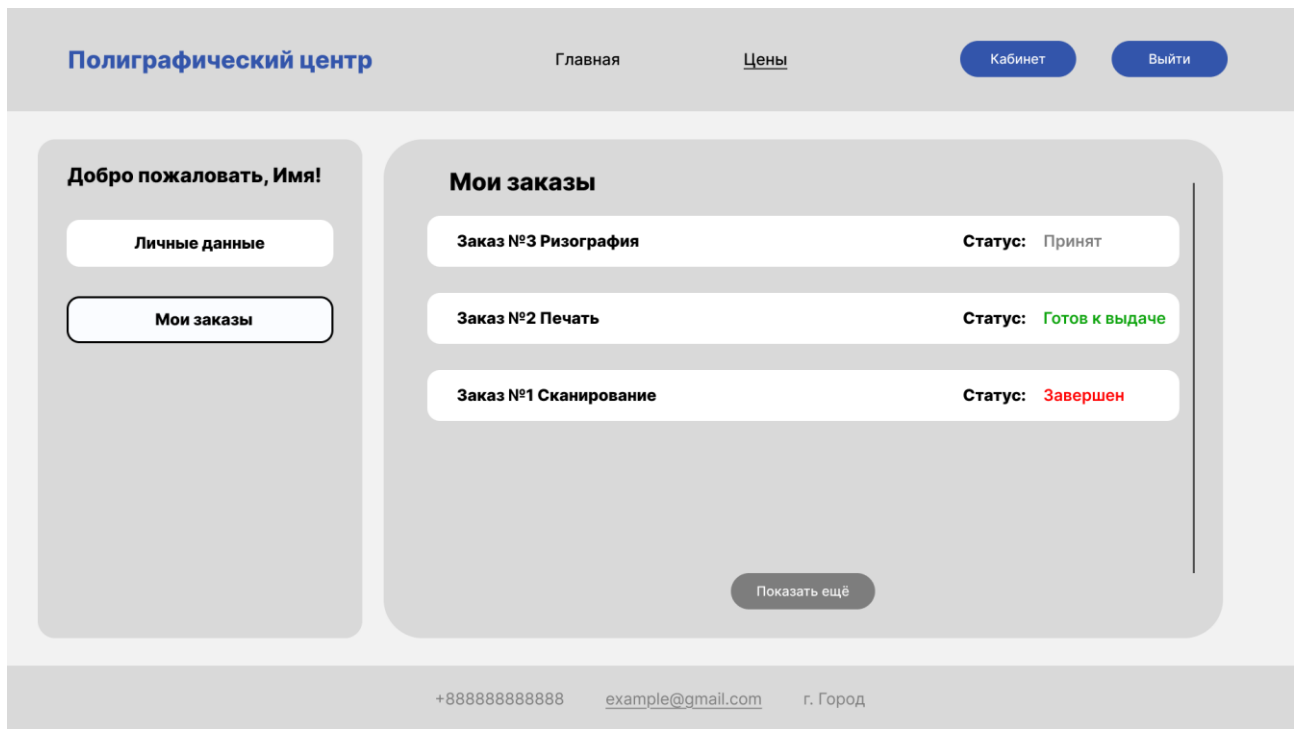


Рисунок 9 - Личный кабинет с открытой вкладкой «Мои заказы»

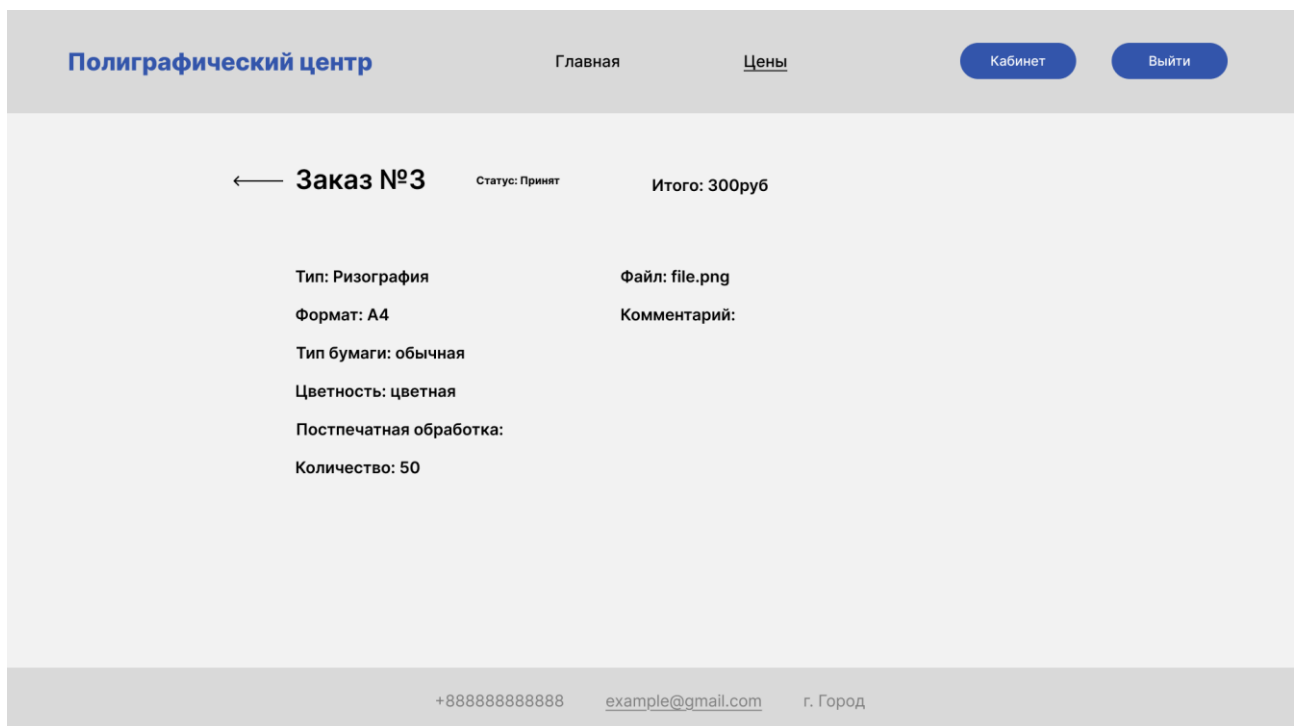


Рисунок 10 — Страница заказа клиента

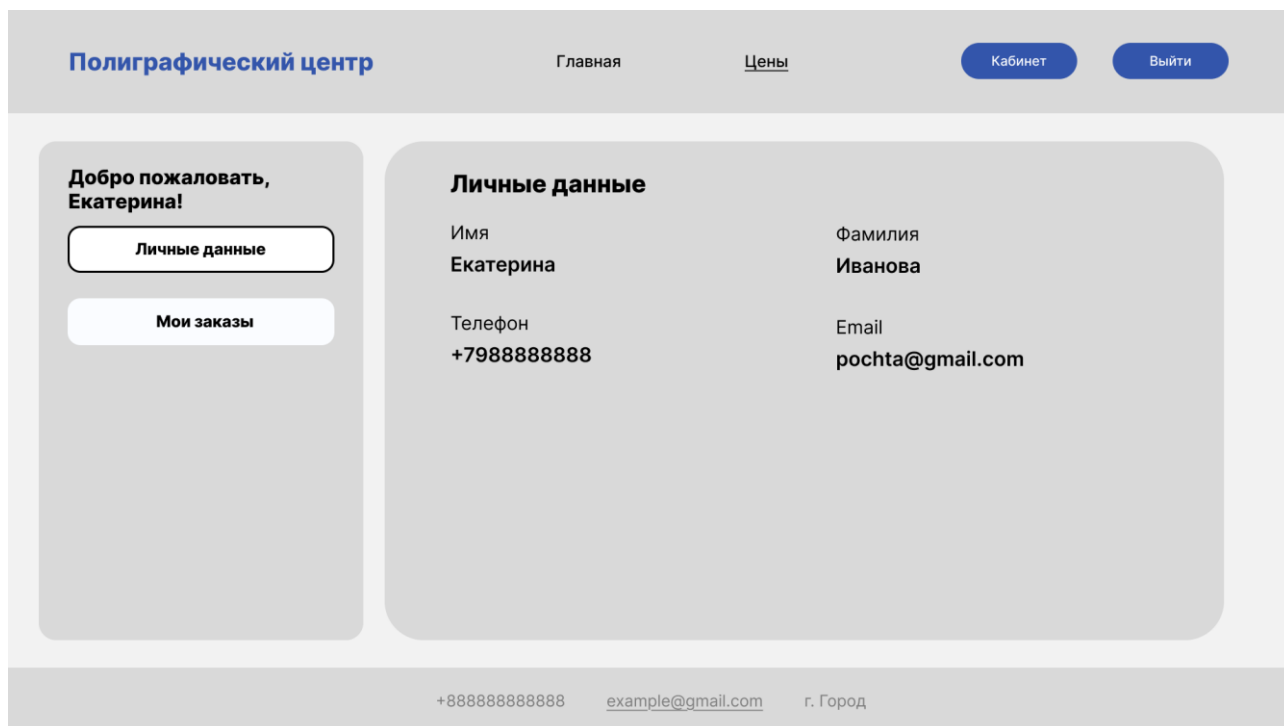


Рисунок 11 — Личный кабинет с открытой вкладкой «Личные данные»

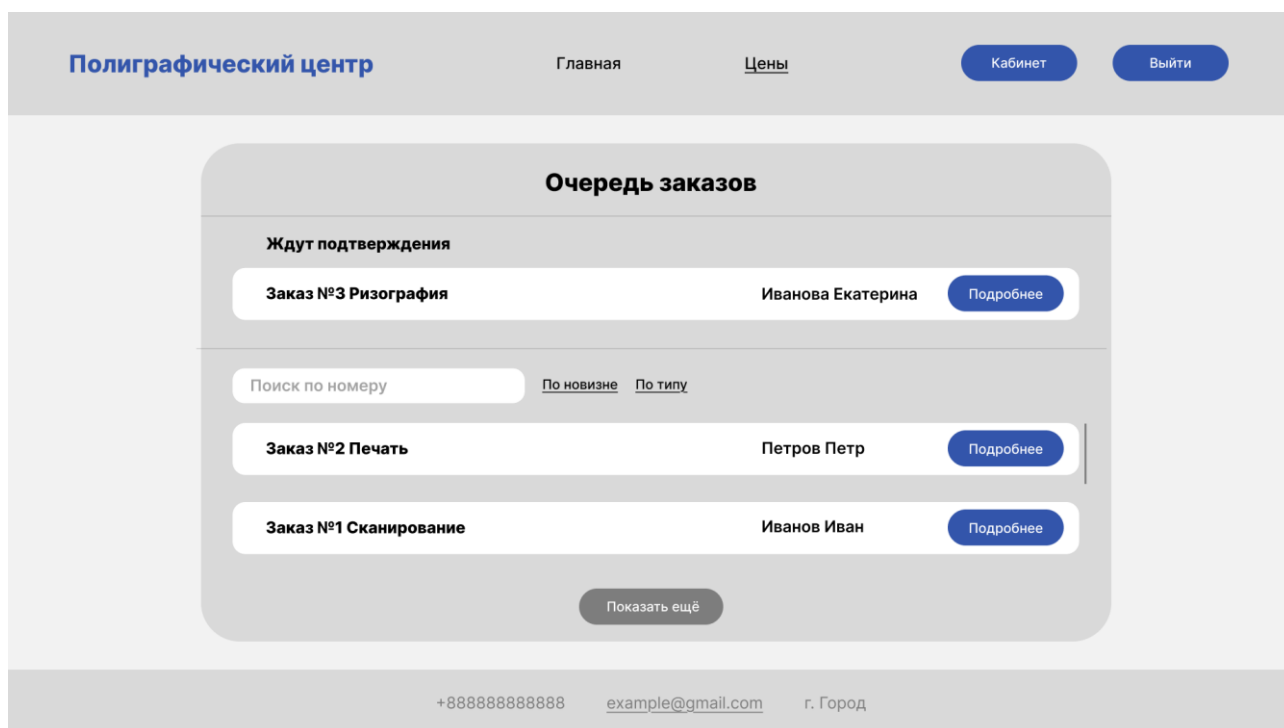


Рисунок 12 — Очередь заказов для сотрудника

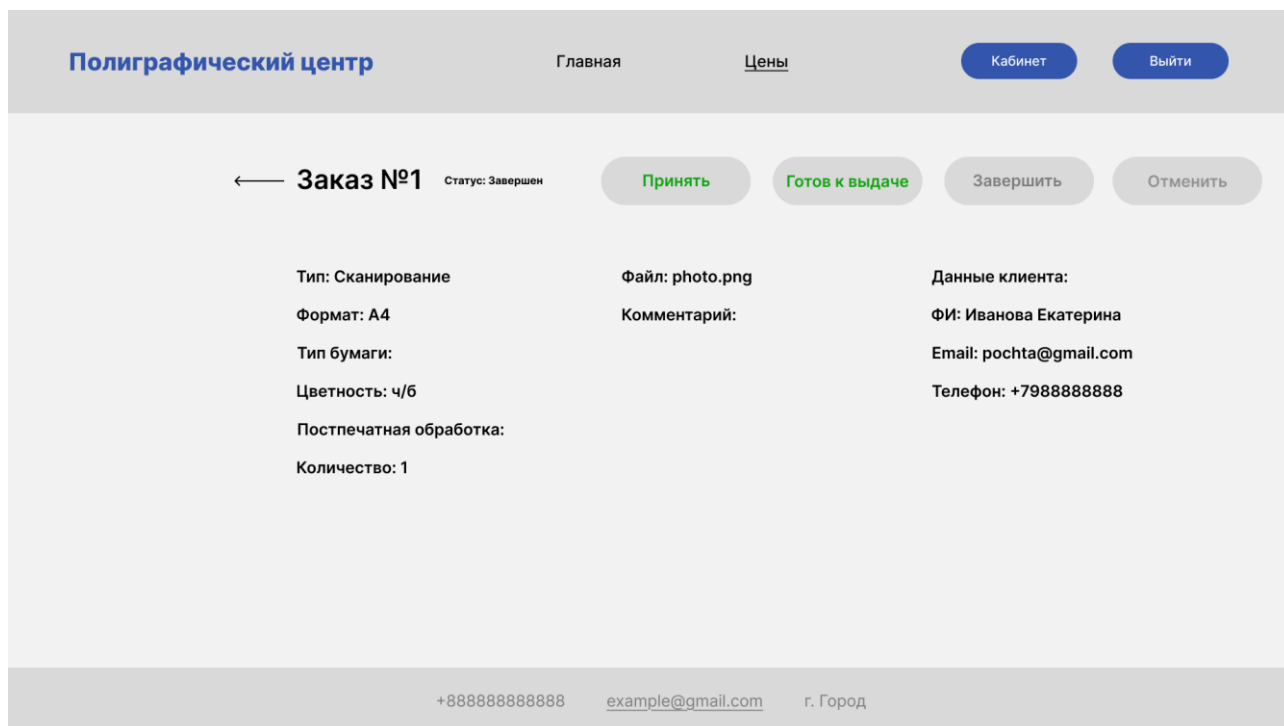


Рисунок 13 — Страница заказа для сотрудника

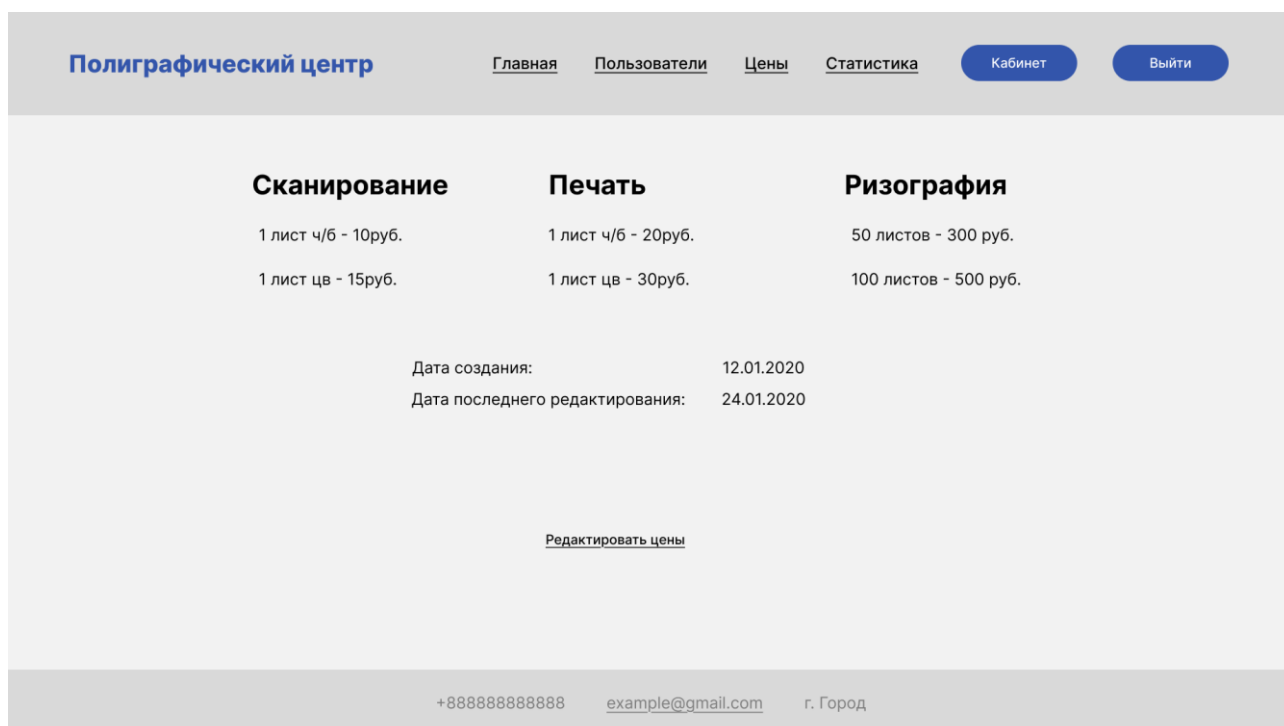


Рисунок 14 — Вкладка «Цены» для администратора

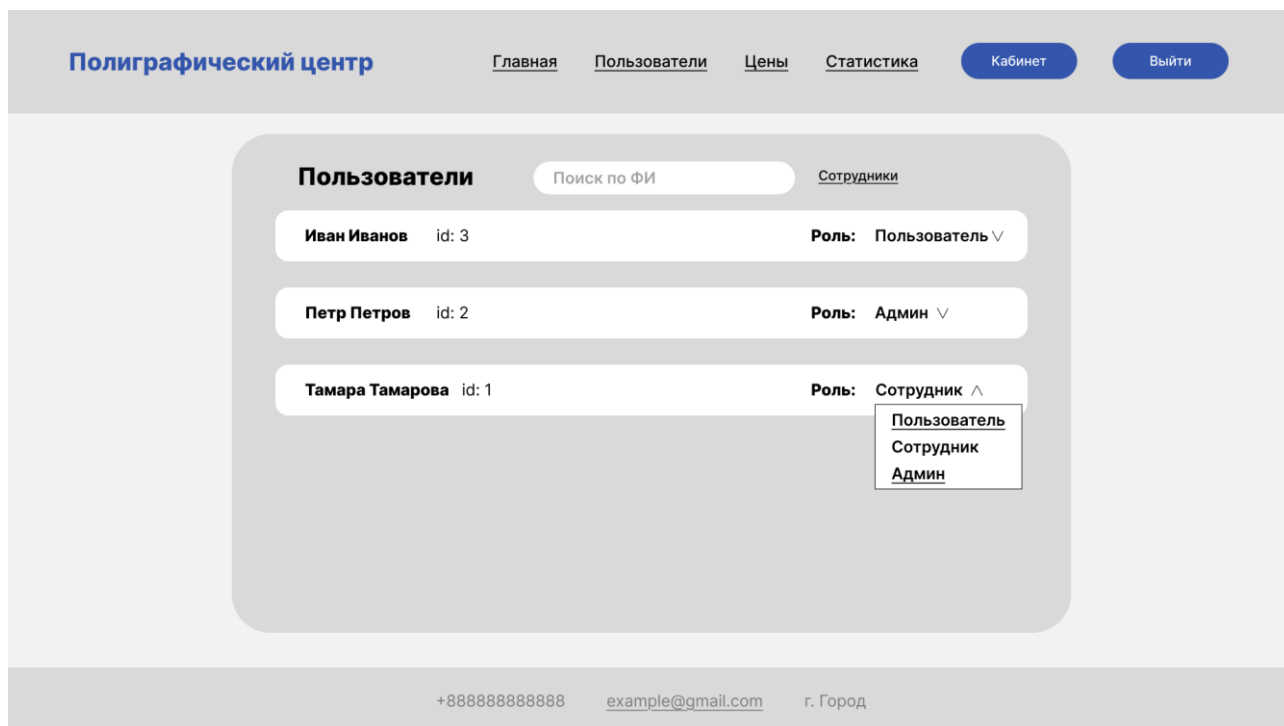


Рисунок 15 — Вкладка «Пользователи» для администратора

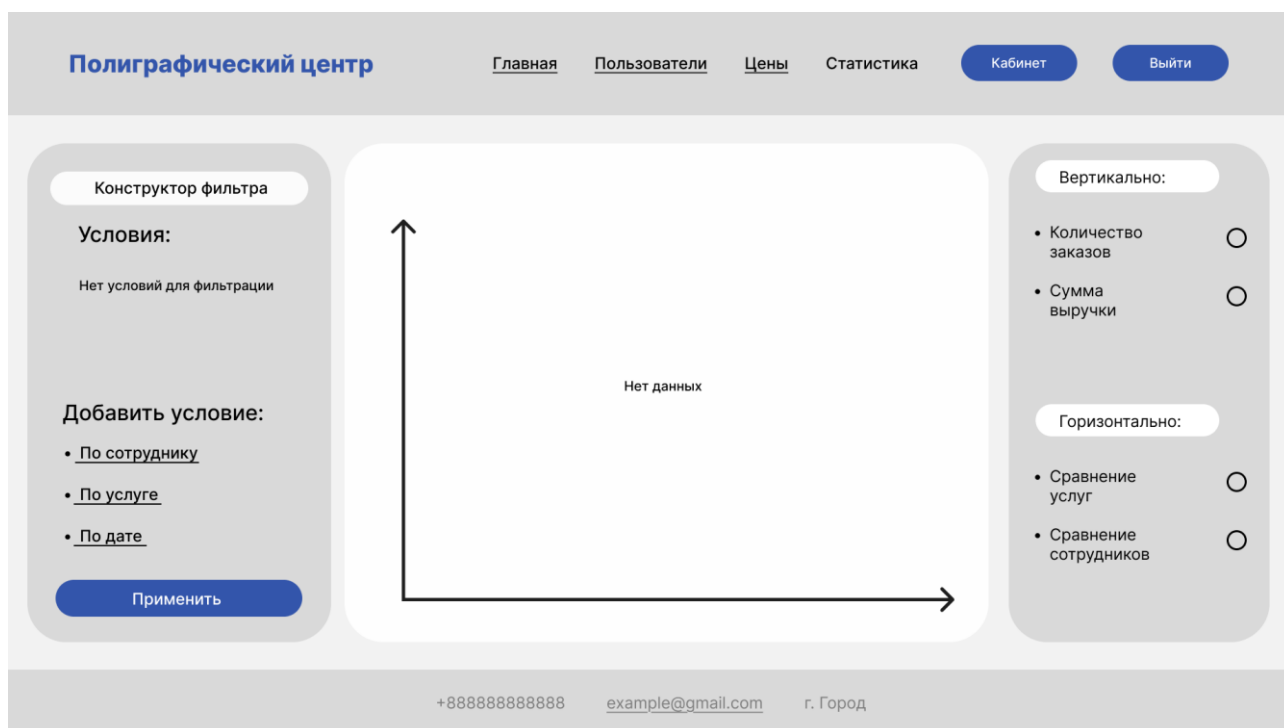


Рисунок 16 — Кастомизированная статистика до построение графика

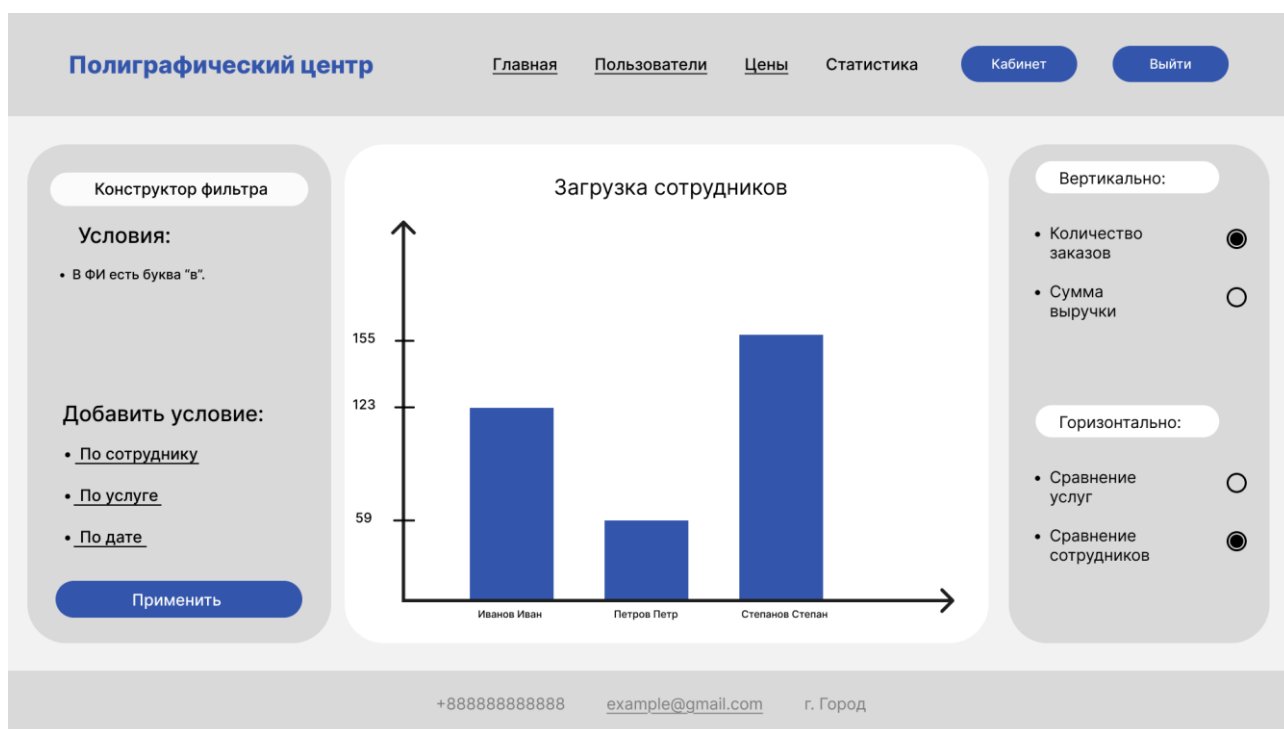


Рисунок 17 — Кастомизированная статистика после построения графиков

Полиграфический центр Главная Пользователи Цены Статистика Кабинет Выйти

Добро пожаловать, Екатерина!

[Личные данные](#)

[Мои заказы](#)

Импорт данных

Пользователи ☐

Заказы ☐

Услуги ☐

[Добавить файл для импорта](#)

Отмена [Импортировать](#)

+8888888888888888 example@gmail.com г. Город

Рисунок 18 — Импорт данных

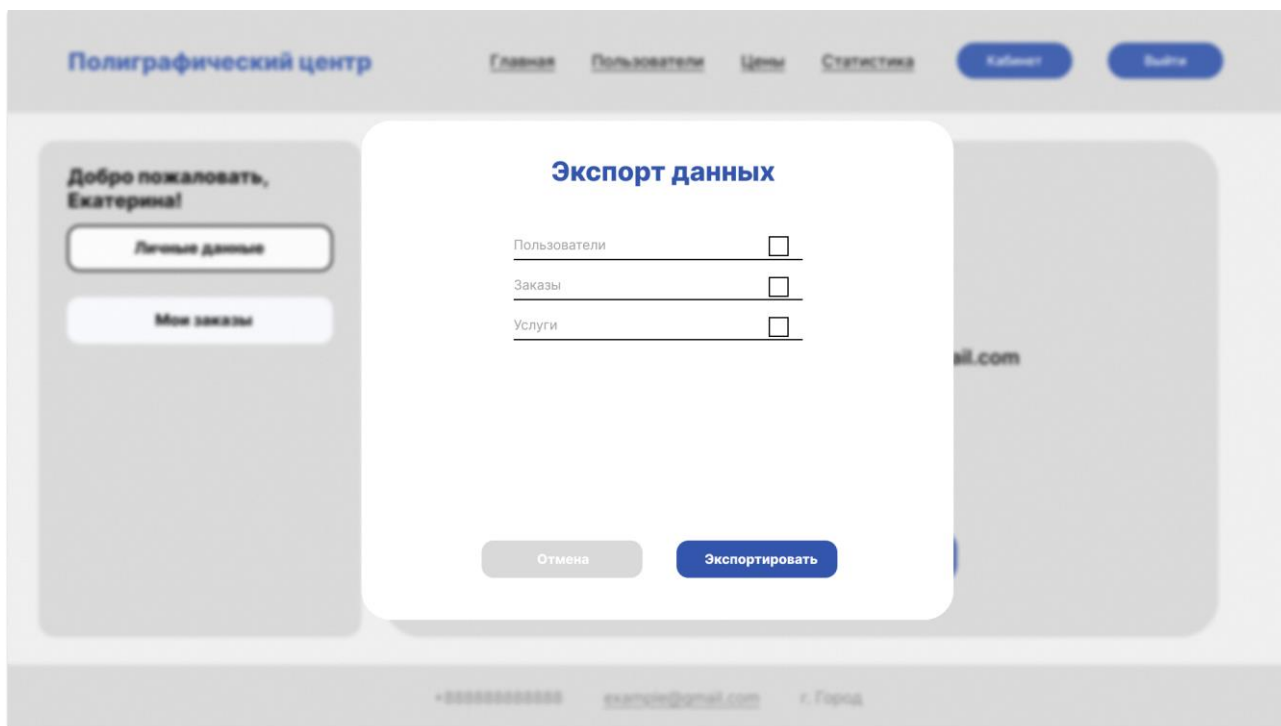


Рисунок 19 — Экспорт данных



Рисунок 20 - Карта экранов и переходов

1.2. Сценарии использования для задачи

Заголовок	Регистрация на сайте
Основное действующее лицо	Клиент, сотрудник
Предусловие	1. Пользователь открыл сайт. 2. Пользователь не имеет актуальной учетной записи.
Результат	Пользователь создает аккаунт с личным кабинетом.
Описание	1. Пользователь нажимает кнопку “Регистрация” 2. Система открывает форму для регистрации 3. Пользователь вводит свои данные, нажимает кнопку “Зарегистрироваться” 4. Система проверяет данные пользователя и наличие пользователя с таким email/номером телефона в системе. 5. Система отображает сообщение об успешной регистрации 6. Пользователь перенаправляется в личный кабинет
Альтернативный сценарий	A1 Ошибка валидации данных на шаге 4 1. Система подсвечивает красным поля, заполненные с ошибкой (например, «Пароли не совпадают» или «Email введен некорректно»). 2. Сценарий возвращается к шагу 3 основного сценария (пользователь исправляет ошибки). A2 Пользователь нажал кнопку “Отмена” на шаге 3 1. Пользователь не зарегистрирован и перенаправляется на главный экран

Заголовок	Оформление заказа полиграфической услуги
Основное	Клиент

действующее лицо	
Предусловие	1. Пользователь открыл сайт и находится на главной странице. 2. Пользователь авторизован в системе.
Результат	Пользователь оформляет заказ.
Описание	1. Пользователь нажимает на кнопку “Заказать” карточки с интересующей его услугой. 2. Система открывает форму для оформления заказа. 3. Пользователь заполняет поля с данными о заказе(формат, количество и т. д.). 4. Выбор файла для услуг печати и ризографии. 1. Пользователь нажимает кнопку «Загрузить файл». 2. Система открывает стандартное окно выбора файлов операционной системы. 3. Пользователь выбирает нужный файл на своем устройстве. 4. Система проверяет файл (формат, размер) и отображает имя файла и иконку рядом с кнопкой загрузки. 5. Пользователь нажимает кнопку “Рассчитать”. 6. Система выводит на экран рассчитанную стоимость. 7. Пользователь нажимает кнопку “Заказать”. 8. Пользователь перенаправляется на страницу созданного заказа.
Альтернативный сценарий	A1 Ошибка при загрузке файла (неверный формат, превышен максимально допустимый размер) на шаге 4d. 1. Система отображает сообщение с информацией об ошибке. 2. Сценарий возвращается к шагу 4 основного сценария.

	A2 Не заполнены обязательные поля на шагах 5 и 7. 1. Система подсвечивает красным незаполненные поля и выводит у каждого поля “Заполните это поле”. 2. Сценарий возвращается к шагу 4 основного сценария. A3 Отмена оформления на шагах от 3 до 7. 1. Пользователь возвращается на главную страницу, все данные о заказе удаляются.
--	--

Заголовок	Авторизация на сайте
Основное действующее лицо	Клиент/сотрудник/администратор
Предусловие	1. Пользователь открыл сайт. 2. Пользователь не авторизован в системе. 3. Пользователь имеет действующую учетную запись
Результат	Пользователь входит в свой аккаунт с личным кабинетом.
Описание	1. Пользователь нажимает кнопку “Вход”. 2. Система открывает форму для входа. 3. Пользователь заполняет поля “Email” и “Пароль”. 4. Пользователь нажимает кнопку “Войти”. 5. Пользователь аутентифицирован и перенаправляется на главную страницу
Альтернативный сценарий	A1 Ошибка валидации данных на шаге 4 1. Система подсвечивает проблемные поля красным и отображает сообщения 2. Сценарий возвращается к шагу 3 основного сценария (пользователь исправляет ошибки). A2. Неверный пароль на шаге 4

	1. Система отображает общее сообщение об ошибке: «Неверный email или пароль». 2. Сценарий возвращается к шагу 3 основного сценария. А3 Пользователь нажал кнопку “Отмена” на шаге 3 2. Пользователь не авторизован и перенаправляется на главный экран
--	--

Заголовок	Просмотр детальной информации о заказе
Основное действующее лицо	Клиент
Предусловие	1. Пользователь открыл сайт. 2. Пользователь авторизован в системе. 3. У пользователя есть хотя бы один оформленный заказ.
Результат	Пользователь просмотрел подробную информацию о заказе.
Описание	1. Пользователь нажимает кнопку “Кабинет”. 2. Пользователь открывает раздел "Мои заказы". 3. Из списка всех заказов выбирает интересующий заказ и нажимает кнопку “Подробнее” 4. Система открывает страницу с детальной информацией о заказе, которая включает параметры заказа, файл и комментарий.
Альтернативный сценарий	-

Заголовок	Обработка заказа
Основное действующее лицо	Сотрудник

Предусловие	1. Сотрудник открыл сайт и находится на главной странице. 2. Сотрудник авторизован в системе. 3. Сотрудник имеет хотя бы один заказ
Результат	Сотрудник изменил статус заказа или с “Принят” на “В очереди”, или с “Принят” на “Отменен”, или с “В очереди” на “Готов к выдаче”
Описание	1. На панели “Очередь заказов” сотрудник нажимает кнопку «Подробнее» рядом с интересующим его заказом. 2. Система открывает модальное окно с полной информацией о заказе: данные клиента (имя, контакты), состав заказа (услуги, параметры), загруженные файлы. 3. Сотрудник нажимает кнопку смены статуса заказа («Подтвердить заказ», “Готов к выдаче”, “Отменить заказ”). 4. Система возвращает сотрудника к списку ожидающих заказов.
Альтернативный сценарий	A1 На шаге 3 сотрудник нажал “Отменить заказ” 1. Система открывает форму для описания причины отмены заказа. 2. Сотрудник нажимает на поле ввода и описывает причину отмены заказа. 3. Сотрудник нажимает кнопку «Подтвердить отклонение» 4. Система возвращает сотрудника к списку ожидающих заказов.

Заголовок	Редактирование стоимости услуги.
Основное действующее лицо	Администратор.

Предусловие	1. Администратор авторизован в системе. 2. Администратор имеет права на редактирование услуг. 3. В системе существует хотя бы одна услуга.
Результат	Стоимость услуги успешно обновлена в системе.
Описание	1. Система отображает список всех услуг. 2. Администратор нажимает кнопку "Редактировать цены". 3. Администратор изменяет стоимость услуги. 4. Администратор нажимает кнопку "Сохранить". 5. Система сохраняет изменения и отображает обновленный список услуг.
Альтернативный сценарий	A1 Ошибка валидации на шаге 4 1. Система подсвечивает проблемные поля красным и отображает сообщения об ошибках около проблемных полей. 2. Сценарий возвращается к шагу 3 основного сценария (пользователь исправляет ошибки).

Заголовок	Выдача роли для нового сотрудника.
Основное действующее лицо	Администратор
Предусловие	1. Администратор авторизован в системе. 2. Администратор имеет права на управление сотрудниками. 3. Администратор находится в разделе "Пользователи". 4. Существует пользователь.
Результат	Пользователю назначена роль "Сотрудник", он получил соответствующие права доступа.
Описание	1. Система отображает таблицу всех

	пользователей с колонками: "ФИ", "id", "Роль" (Пользователь/Сотрудник/Админ). 2. Администратор находит нужного пользователя в списке. 3. Администратор в колонке "Роль" нажимает на текущее значение (выпадающий список). 4. Система отображает доступные роли: "Пользователь", "Сотрудник", "Администратор". 5. Администратор выбирает роль "Сотрудник". 6. Система отображает обновленный список пользователей с измененной ролью.
Альтернативный сценарий	A1 Пользователь уже имеет роль "Сотрудник" 1. Система отображает текущую роль

Заголовок	Построение кастомизированной статистики.
Основное действующее лицо	Администратор
Предусловие	1. Администратор авторизован в системе 2. Администратор имеет права на просмотр статистики 3. В системе существуют данные для анализа.
Результат	Сформирована диаграмма на основе заданных критериев фильтрации и выбранных осей визуализации.
Описание	1. Администратор открывает раздел "Статистика". 2. Администратор задает фильтр для отбора данных, можно несколько условий (по сотруднику (возраст и ФИ)/по услуге (количество и цена)/по дате (начало и конец)): Администратор нажимает на кнопку с

	условием ("По сотруднику"/"По услуге"/"По дате"). • Система отображает модальное окно. • Администратор вводит данные для фильтров. • Администратор нажимает кнопку "Применить". • Система закрывает модальное окно. • Система отображает обновленный список условий. 3. Администратор выбирает, что показать на диаграмме по оси X (услуги/сотрудники). 4. Администратор выбирает, что показать на диаграмме по оси Y (количество/сумма). 5. Администратор нажимает кнопку "Применить". 6. Система фильтрует данные, группирует и строит столбчатую диаграмму
Альтернативный сценарий	A1 Нет данных по фильтру на шаге 5. 1. Система показывает "Нет данных" 2. Сценарий возвращается к шагу 2. A2 Администратор нажимает кнопку "Отменить" шаге 2. 1. Система закрывает модальное окно 2. Сценарий возвращается к шагу 2.

Заголовок	Импорт данных.
Основное действующее лицо	Администратор
Предусловие	1. Администратор авторизован в системе. 2. Администратор имеет права на импорт данных. 3. Подготовлен файл с данными в поддерживаемом формате. 4. Администратор находится в "Кабинете" во

	вкладке "Личные данные".
Результат	Данные успешно импортированы в систему.
Описание	1. Администратор нажимает кнопку "Импортировать данные". 2. Система отображает модальное окно импорта. 3. Администратор выбирает тип импортируемых данных (пользователи/услуги/заказы). 4. Администратор нажимает кнопку "Добавить файл для импорта". 5. Администратор загружает файл с данными. 6. Администратор нажимает кнопку "Импортировать". 7. Система отображает сообщение об успешном импорте данных.
Альтернативный сценарий	A1 Неверный формат файла на шаге 6 1. Система отображает сообщение об ошибке "Неподдерживаемый формат файла". 2. Система предлагает загрузить файл в нужном формате. 3. Сценарий возвращается к шагу 5. A2 Ошибки отсутствия типа импортируемых данных на шаге 6 1. Сценарий возвращается к шагу 3. A3 Администратор отменил импорт на шаге 5 1. Система не импортирует данные. 2. Система возвращается к предыдущему экрану.

Заголовок	Экспорт данных.
Основное действующее лицо	Администратор
Предусловие	1. Администратор авторизован в системе.

	2. Администратор имеет права на экспорт данных. 3. В системе существуют данные для экспорта. 4. Администратор находится в "Кабинете" во вкладке "Личные данные".
Результат	Данные экспортированы в файл и файл скачан на устройство администратора.
Описание	1. Администратор нажимает кнопку "Экспортировать данные". 2. Система отображает модальное окно экспорта. 3. Администратор выбирает тип экспортируемых данных (пользователи/услуги/заказы). 4. Администратор нажимает кнопку "Экспортировать". 5. Система отображает сообщение об успешном экспорте данных.
Альтернативный сценарий	A1 Нет данных для экспорта на шаге 4 1. Система отображает сообщение об ошибке "Нет данных для экспорта по заданным параметрам". 2. Система предлагает изменить тип данных. 3. Сценарий возвращается к шагу 3. A2 Администратор отменил экспорт на шаге 4 3. Система не экспортирует данные. 4. Система возвращается к предыдущему экрану.

1.3. Вывод о преобладающих операциях

Проанализировав сценарии использования информационной системы центра полиграфических услуг, можно сделать вывод, что характер нагрузки на базу данных Neo4j будет смешанным, с незначительным преобладанием операций чтения над записью.

2. МОДЕЛЬ ДАННЫХ

2.1. Нереляционная модель данных

2.1.1 Графическое представление

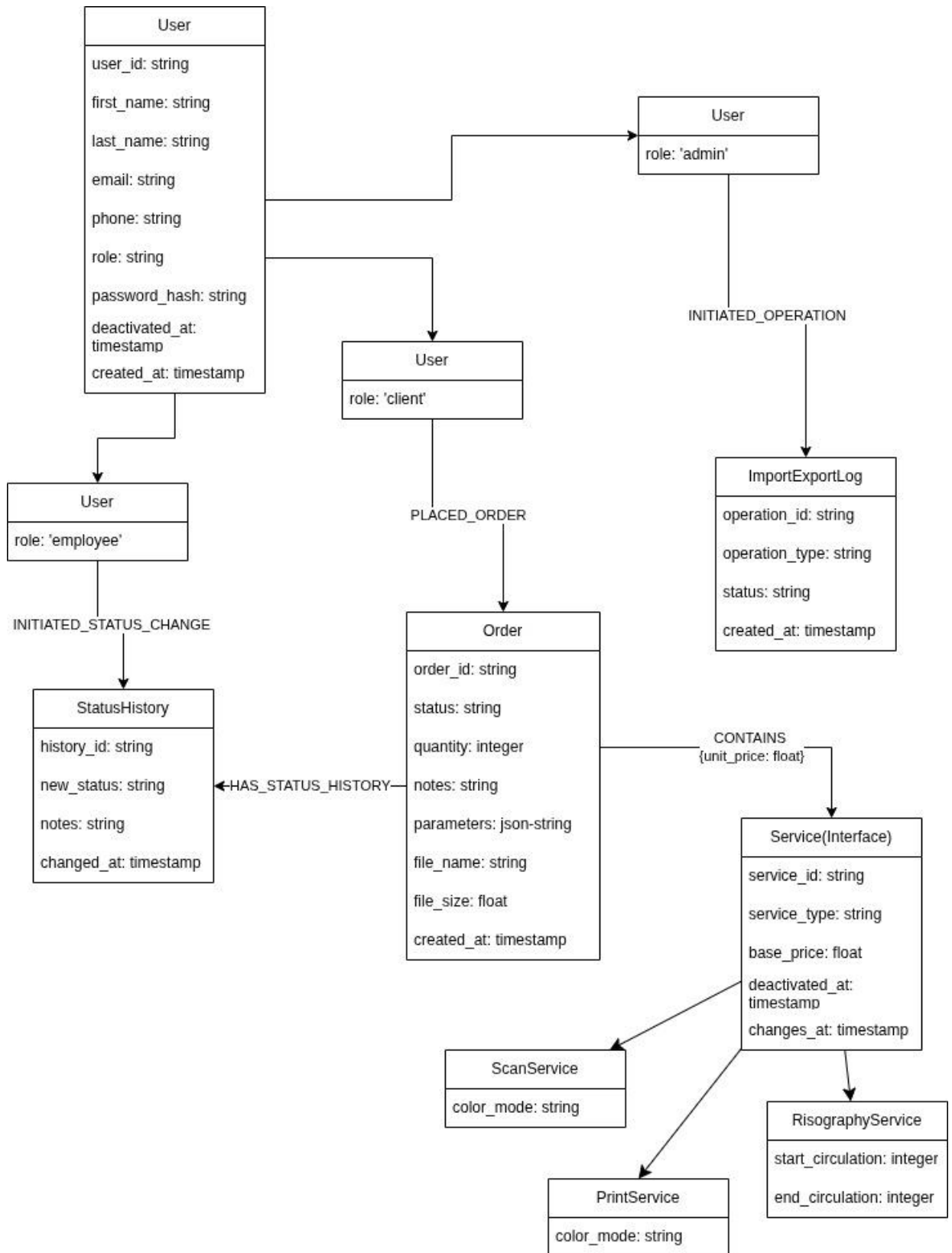


Рисунок 21 — Графическое представление нереляционной базы данных

2.1.2 Описание назначений коллекций, типов данных и сущностей

Структура записи свойства (41 байт)

Запись свойства состоит из двух частей: служебного заголовка и полезной нагрузки (payload).

Заголовок (9 байт): Содержит флаг использования, идентификатор типа данных (4 бита для ключа и 24 бита для типа) и указатель на следующую запись свойства в цепочке.

Payload (32 байта): Разделен на четыре блока по 8 байт.

Примитивные типы (boolean, int, float, timestamp) занимают часть payload. Если строка не помещается в 32 байта полезной нагрузки, Neo4j использует файлы динамического хранения- neostore.propertystore.db.strings. В этом случае в 41-байтовой записи свойства сохраняется ссылка (указатель) на адрес первого блока в динамическом хранилище. Динамическое хранилище оперирует блоками фиксированного размера — 128 байт. Строка квантуется на эти блоки.

User (Пользователь)

Поле	Тип данных	Размер (байт)	Описание
user_id	string	169(41+128)	Уникальный идентификатор пользователя, состоящий из 36 символов
email	string	297(41+256)	Уникальный адрес электронной почты, используемый для входа в систему
password_hash	string	169(41+128)	Хеш пароля (например, bcrypt).
first_name	string	169(41+128)	Имя пользователя
last_name	string	169(41+128)	Фамилия пользователя
phone	string	41	Номер телефона (опционально)
role	string	41	Роль пользователя, определяющая права и доступ к

			функциям системы ('admin', 'employee', 'client')
created_at	timestamp	41	Дата и время создания учетной записи
deactivated_at	timestamp	41	Дата и время деактивации аккаунта (NULL = аккаунт активен)

Order (Заказ)

Поле	Тип данных	Размер (байт)	Описание
order_id	string	169(41+128)	Уникальный идентификатор заказа
quantity	integer	41	Количество единиц услуги (например, количество листов или страниц)
parameters	string	169(41+128)	Параметры заказа в формате JSON-строки
notes	string	169(41+128)	Комментарий клиента к заказу (опционально)
status	string	41	Текущий статус заказа ('pending', 'processing', 'ready', 'cancelled', 'completed')
created_at	timestamp	41	Дата и время создания заказа
file_name	string	297(41+256)	Оригинальное имя загруженного файла
file_size	integer	41	Размер файла в байтах

ScanService и PrintService (Услуги печати и сканирования)

Поле	Тип данных	Размер (байт)	Описание
------	------------	---------------	----------

service_id	string	169(41+128)	Уникальный идентификатор услуги
service_type	string	41	Тип услуги ('scan', 'print')
base_price	float	41	Текущая базовая стоимость услуги
deactivated_at	timestamp	41	Дата и время деактивации услуги (NULL = аккаунт активен)
changed_at	timestamp	41	Дата и время последнего обновления стоимости услуги
color_mode	string	41	Услугу выполнять в каком цвете ("color или "bw")

RisographyService (Услуга ризографии)

Поле	Тип данных	Размер (байт)	Описание
service_id	string	169(41+128)	Уникальный идентификатор услуги
service_type	string	41	Тип услуги ('risography')
base_price	float	41	Текущая базовая стоимость услуги
deactivated_at	timestamp	41	Дата и время деактивации услуги (NULL = аккаунт активен)
changed_at	timestamp	41	Дата и время последнего обновления стоимости услуги
start_circulation	integer	41	От какого числа тиража актуальна цена
end_circulation	integer	41	До какого числа тиража актуальна цена

StatusHistory (История статусов заказа)

Поле	Тип данных	Размер (байт)	Описание
history_id	string	169(41+128)	Уникальный идентификатор истории изменения статуса
new_status	string	41	Новый статус после изменения ('pending', 'processing', 'ready', 'cancelled', 'completed')
changed_at	timestamp	41	Дата и время изменения статуса
comment	string	297(41+256)	Описание причины отмены заказа (опционально)

ImportExportLogs (Журнал импорта/экспорта)

Поле	Тип данных	Размер (байт)	Описание
log_id	string	169(41+128)	Уникальный идентификатор операции импорта/экспорта
operation_type	string	41	Тип операции('import', 'export')
status	string	41	Статус операции ('success', 'failed')
created_at	timestamp	41	Дата и время выполнения операции

Служебная информация каждого узла занимает 16 байт(флаги, id первой связи, id первого свойства, данные о метках)

Связи:

- PLACED_ORDER - связь между клиентом и заказом;
- CONTAINS - связь между заказом и услугой, которую содержит заказ(имеет атрибут unit_price, занимающий 41 байт, хранящий информацию о цене единицы продукции в момент оформления заказа);
- HAS_STATUS_HISTORY - связь между заказом и историей изменения статуса заказа;

- INITIATED_STATUS_CHANGE - связь между историей изменения статуса заказа и сотрудником, который изменил статус заказа;
- INITIATED_OPERATION - связь между операцией импорта/экспорта и администратором, который ее инициировал.

Служебная информация связи занимает 34 байт.

2.1.3 Оценка удельного объема информации, хранимой в модели

Средний размер одного узла

User: $169 + 297 + 169 + 169 + 169 + 41 + 41 + 41 + 41 + 41 + 16(\text{служебка}) = 1153$ байт

Order: $169 + 41 + 169 + 169 + 41 + 41 + 297 + 41 + 16$ (служебка) = 984 байт

ScanService и PrintService: $169 + 41 + 41 + 41 + 41 + 41 + 16(\text{служебка}) = 390$ байт

RisographyService: $169 + 41 + 41 + 41 + 41 + 41 + 41 + 16(\text{служебка}) = 431$ байт

StatusHistory: $169 + 41 + 41 + 297 + 16(\text{служебка}) = 564$ байт

ImportExportLogs: $169 + 41 + 41 + 41 + 16$ (служебка) = 308 байт

Средний размер связей

- PLACED_ORDER: 34 байт
- CONTAINS: $34 + 41 = 75$ байт
- HAS_STATUS_HISTORY: 34 байт
- INITIATED_STATUS_CHANGE: 34 байт
- INITIATED_OPERATION: 34 байт

Коэффициенты зависимости от количества пользователей

Переменная	Значение	Описание
U	переменная	Количество пользователей (базовая переменная)

R_orders	5	Среднее количество заказов на одного пользователя
R_statuschanges	3	Среднее количество изменений статуса на один заказ
D_employees	0.05	Доля сотрудников среди пользователей (5%)
D_admins	0.01	Доля администраторов среди пользователей (1%)
R_impexp	10	Среднее количество операций импорта/экспорта на одного администратора
SP_services	4	Количество услуг scan/print в справочнике (константа)
R_services	2	Количество услуг risography в справочнике (константа)
R_contains	1	Среднее количество услуг на один заказ (каждый заказ содержит одну услугу)
R_history_links	1	Связь HAS_STATUS_HISTORY на каждую запись истории
R_status_initiator	1	Доля записей истории, у которых указан сотрудник
R_user_orders	1	Один пользователь создает заказ (для PLACED_ORDER)
R_log_admin	1	Каждая операция связана с одним администратором

Формула общего объема данных

User: $1153 \times U = 1153U$

Order: $984 \times (U \times R_orders) = 984 \times 5U = 4920U$

ScanPrintService: $390 \times SP_services = 390 \times 4 = 1560$ (константа)

RisoService: $431 \times R_services = 431 \times 2 = 862$ (константа)

OrderStatusHistory: $564 \times (U \times R_orders \times R_statuschanges) = 564 \times 5 \times 3 \times U = 8460U$

ImportExportLogs: $308 \times (U \times D_admins \times R_impexp) = 308 \times 0,01 \times 10 \times U = 30,8U$

Объем связей

PLACED_ORDER: $34 \times R_orders \times R_userorders \times U = 34 \times 5U = 170U$

CONTAINS: $75 \times R_orders \times R_contains \times U = 75 \times 5U = 375U$

HAS_STATUS_HISTORY: $34 \times R_orders \times R_statuschanges \times R_historylinks \times U = 34 \times 15U = 510U$

INITIATED_STATUS_CHANGE: $34 \times R_orders \times R_statuschanges \times R_statusinitiator = 34 \times 15U = 510U$

INITIATED_OPERATION: $34 \times D_admins \times R_impexp \times R_logadmin = 34 \times 0,1U = 3,4U$

Итоговая формула зависимости объема от количества пользователей

Узлы: $V_actual(U) = 1153U + 4920U + 8460U + 30,8U + 1560 + 862$

Связи: $V_actual(U) = 170U + 375U + 510U + 510U + 3,4U = 1568,4U$

Итого: $V_actual^{neo4j}(U) = 16132,2U + 2422$

Избыточность данных

Избыточность модели данных — это отношение между фактическим объемом модели и «чистым» объемом данных.

Фактический объем модели: $V_actual^{neo4j}(U) = 16132,2U + 2422$

«Чистый» объем данных:

User (email + first_name + last_name + password_hash + phone + role): $257 + 102 + 102 + 60 + 22 + 1 = 544 \Rightarrow 544U$

Order (quantity + parameters + unit_price + note + status + file_name + file_size): $2 + 100 + 12 + 100 + 1 + 257 + 4 = 476 \Rightarrow 476 \times 5U = 2380U$

Service (service_name + base_price + description + service_type): $52 + 10 + 200 + 1 = 263 \Rightarrow 263 \times 6 = 1578$ (константа)

Order_Status_History (new_status + comment): $1 + 200 = 201 \Rightarrow 201 \times 15U = 3015U$

Import_Export_Logs (operation_type + status): $1 + 1 = 2 \Rightarrow 2 \times 0,1U = 0,2U$

$V_{\text{clean}}(U) = 544U + 2380U + 1578 + 3015U + 0,2U$

$V_{\text{clean}}(U) = 5939,2U + 1578$

Формула избыточности

$\text{Redundancy}(U) = V_{\text{actual}}^{\text{neo4j}}(U) / V_{\text{clean}}(U) = (16132,2U + 2422) / (5939,2U + 1578)$

При больших U $\text{Redundancy} \approx 16132,2 / 5939,2 \approx 2,72$

2.1.4 Запросы к модели, с помощью которых реализуются сценарии

использования

Для всех лейблов обеспечиваем уникальность id. Пример для лейбла User

```
CREATE CONSTRAINT unique_user_id IF NOT EXISTS FOR (u:User) REQUIRE
u.user_id
```

Сценарий 1: Регистрация на сайте

```
MATCH (u:User {email: 'katya.lobova2005@gmail.com'})
```

```
RETURN count(u) > 0 AS exists
```

```
CREATE (u:User {
  user_id: randomUUID(),
  first_name: 'Ekaterina',
  last_name: 'Lobova',
  email: 'katya.lobova2005@gmail.com',
  phone: '+79210702386',
  role: 'admin',
  deactivated_at: null,
  created_at: datetime(),
  password_hash: '$2y$10$abc123...'
```

```
})  
RETURN u
```

Количество запросов: 2

Задействованные узлы одного типа: 1 (User)

Сценарий 2: Авторизация на сайте

```
-- Проверка учетных данных  
MATCH (u:User {email: "katya.lobova2005@gmail.com"})  
WHERE u.password_hash = $password_hash  
  AND u.deactivated_at IS NULL  
RETURN u.user_id AS user_id,  
       u.first_name AS first_name,  
       u.last_name AS last_name,  
       u.email AS email,  
       u.role AS role,  
       u.created_at AS created_at
```

Количество запросов: 1

Задействованные узлы одного типа: 1 (User)

Сценарий 3: Оформление заказа полиграфической услуги

```
//Получение базовой стоимости услуги  
MATCH (s:Service)  
WHERE s.service_type = "print" AND s.color_mode = "color"  
RETURN s.base_price, s.service_id  
  
// Создаём заказ и сразу сохраняем его в переменную  
CREATE (o1:Order {  
  order_id: randomUUID(),  
  parameters: '{format: "A4", color_mode: "color", double_sided: true}',  
  quantity: 15,
```

```

status: "pending",
notes: "Срочно, к 15:00",
file_name: "document.pdf",
file_size: 2048576,
created_at: datetime()
})
WITH o1

// Находим пользователя
MATCH (u:User {user_id: "613dd16f-3d83-4959-b5c4-787c6bd12861"})
WITH o1, u

// Находим сервис
MATCH (s:Service {service_id: "a2279b07-c4c3-4d0a-9144-f06d0cbb327a"})
WITH o1, u, s

// Создаём все связи
CREATE (u)-[:PLACED_ORDER]->(o1)
CREATE (o1)-[:CONTAINS {
  unit_price: s.base_price
}]->(s)

RETURN o1, u, s

```

Количество запросов: 2

Задействованные узлы одного типа: 3 (User, Service, Order)

Сценарий 4: Просмотр детальной информации о заказе

```

// Все заказы пользователя
MATCH (u:User {user_id: "613dd16f-3d83-4959-b5c4-787c6bd12861"})-
[:PLACED_ORDER]->(o:Order)

```



```
OPTIONAL MATCH (o)-[rel:CONTAINS]->(s:Service)
```

```
RETURN
```

```
o.order_id AS order_id,  
o.quantity AS quantity,  
rel.unit_price AS unit_price,  
o.notes AS comment,  
o.file_name AS file_name,  
s.service_type AS service_type,  
o.status AS status,  
o.created_at AS created_at
```

```
ORDER BY o.created_at DESC
```

```
// Получение заказа с его историей
```

```
MATCH (u:User {user_id:"613dd16f-3d83-4959-b5c4-787c6bd12861"})-  
[:PLACED_ORDER]->(o:Order {order_id: "72827686-df86-4e74-877f-  
ffc85b9e28ab"})
```

```
OPTIONAL MATCH (o)-[rel:CONTAINS]->(s:Service)
```

```
OPTIONAL MATCH (o)-[:HAS_STATUS_HISTORY]->(h:StatusHistory)  
    <-[:INITIATED_STATUS_CHANGE]-(e:User)
```

```
RETURN
```

```
u.user_id AS user_id,  
u.first_name AS user_first_name,  
u.last_name AS user_last_name,  
o.order_id AS order_id,  
o.parameters AS parameters,  
o.quantity AS quantity,  
rel.unit_price AS unit_price,  
o.notes AS notes,  
o.file_name AS file_name,
```

```

o.file_size AS file_size,
o.created_at AS order_created_at,
s.service_type AS service_type,
o.status AS status,
collect({
  history_id: h.history_id,
  changed_at: h.changed_at,
  notes: h.notes,
  new_status: h.new_status,
  employee_id: e.user_id,
  employee_name: e.first_name + ' ' + e.last_name
}) AS status_history

```

Количество запросов: 2

Задействованные узлы одного типа: 4 (User, Service, Order, StatusHistory)

Сценарий 5: Обработка заказа

```

// Получение списка заказов в очереди для сотрудника
MATCH (o:Order {status: 'pending'})<-[:PLACED_ORDER]-(u:User)
OPTIONAL MATCH (o)-[:CONTAINS]->(s:Service)
RETURN
  o.order_id AS order_id,
  u.user_id AS user_id,
  u.first_name AS first_name,
  u.last_name AS last_name,
  u.phone AS phone,
  s.service_id AS service_id,
  s.service_type AS service_type,
  o.parameters AS parameters,
  o.file_name AS file_name,
  o.file_size AS file_size,

```

```

o.status AS status,
o.created_at AS created_at
ORDER BY o.created_at ASC

// Смена статуса и добавление записи истории
MATCH (o:Order {order_id: "72827686-df86-4e74-877f-ffc85b9e28ab"})
SET o.status = 'cancelled'
WITH o
OPTIONAL MATCH (e:User {user_id: "cd5a1426-340d-4f1e-88db-6787bfff45f8",
role: "employee"})
CREATE (h:StatusHistory {
  history_id: randomUUID(),
  new_status: 'cancelled',
  changed_at: datetime(),
  notes: "Битый файл, попробуйте заново сохранить файл и отправить"
})
CREATE (o)-[:HAS_STATUS_HISTORY]->(h)
FOREACH (x IN CASE WHEN e IS NOT NULL THEN [1] ELSE [] END |
  CREATE (h)-[:INITIATED_STATUS_CHANGE]->(e)
)
RETURN
o.order_id,
o.status,
h.history_id

```

Количество запросов: 2

Задействованные узлы одного типа: 4 (User, Service, Order, StatusHistory)

Сценарий 6: Редактирование стоимости услуги

```

// Получение списка всех услуг
MATCH (s:Service)

```

```

RETURN
  s.service_id AS service_id,
  s.service_type AS service_type,
  s.base_price AS base_price,
  s.deactivated_at AS deactivated_at,
  s.changed_at AS changed_at
ORDER BY s.service_type ASC

// Обновление и возврат обновлённой услуги
MATCH (s:Service {service_id: "a2279b07-c4c3-4d0a-9144-f06d0cbb327a"})
SET s.base_price = 6.00,
    s.changed_at = datetime()
RETURN
  s.service_id AS service_id,
  s.name AS service_name,
  s.base_price AS base_price,
  s.changed_at AS changed_at

```

Количество запросов: 2

Задействованные узлы одного типа: 1 (Service)

Сценарий 7: Выдача роли для нового сотрудника

```

// Список всех сотрудников
MATCH (u:User)
RETURN
  u.user_id AS user_id,
  u.first_name AS first_name,
  u.last_name AS last_name,
  u.email AS email,
  u.role AS role,
  u.deactivated_at AS deactivated_at

```

```
ORDER BY u.created_at DESC
```

```
// Обновление роли и возврат обновления
```

```
MATCH (u:User {user_id: "613dd16f-3d83-4959-b5c4-787c6bd12861"})
```

```
SET u.role = 'employee'
```

```
RETURN
```

```
  u.user_id AS user_id,
```

```
  u.first_name AS first_name,
```

```
  u.last_name AS last_name,
```

```
  u.role AS role
```

Количество запросов: 2

Задействованные узлы одного типа: 1 (User)

Сценарий 8: Построение кастомизированной статистики

```
// Статистика по услугам за период
```

```
MATCH (o:Order)-[rel:CONTAINS]->(s:Service)
```

```
WHERE o.created_at >= datetime('2025-02-01')
```

```
  AND o.created_at <= datetime('2025-02-28')
```

```
  AND o.status IN ['completed']
```

```
RETURN
```

```
  s.service_type AS service_type,
```

```
  COUNT(o) AS orders_count,
```

```
  SUM(o.quantity * rel.unit_price) AS total_revenue
```

```
ORDER BY total_revenue DESC
```

```
// Статистика по сотрудникам за период
```

```
MATCH (h:StatusHistory)-[:INITIATED_STATUS_CHANGE]->(u:User)
```

```
MATCH (o:Order)-[:HAS_STATUS_HISTORY]->(h)
```

```
MATCH (o:Order)-[rel:CONTAINS]->(s:Service)
```

```
WHERE h.changed_at >= datetime('2026-03-26')
```

```

AND h.changed_at <= datetime('2026-04-28')
AND u.role = 'employee'
RETURN
u.first_name AS first_name,
u.last_name AS last_name,
COUNT(h) AS orders_processed,
SUM(o.quantity * rel.unit_price) AS total_revenue
ORDER BY orders_processed DESC

```

Количество запросов: 2

Задействованные узлы одного типа: 4 (Order, StatusHistory, User, Service)

Сценарий 9: Импорт данных

```
// Начало транзакции (в Neo4j транзакция начинается автоматически)
```

```
// Создание пользователей
```

```

CREATE (u1:User {
  email: 'import1@mail.ru',
  password_hash: '$2y$10$...',
  phone: '+79310602289',
  first_name: 'Импорт',
  last_name: 'Первый',
  role: 'client',
  created_at: datetime(),
  deactivated_at: null,
  user_id: randomUUID()
})

```

```

CREATE (u2:User {
  email: 'import2@mail.ru',
  phone: '+79310602288',
  password_hash: '$2y$10$...',

```

```
first_name: 'Импорт',  
last_name: 'Второй',  
role: 'client',  
created_at: datetime(),  
deactivated_at: null,  
user_id: randomUUID()  
})
```

```
CREATE (u3:User {  
  email: 'import3@mail.ru',  
  phone: '+79310602287',  
  password_hash: '$2y$10$...',  
  first_name: 'Импорт',  
  last_name: 'Третий',  
  role: 'client',  
  created_at: datetime(),  
  deactivated_at: null,  
  user_id: randomUUID()  
})
```

```
WITH [u1, u2, u3] AS users
```

```
MATCH (a:User {user_id: "613dd16f-3d83-4959-b5c4-787c6bd12861"})
```

```
// Создание лога импорта с вычислением статуса
```

```
CREATE (l:ImportExportLog {  
  log_id: randomUUID(),  
  operation_type: 'import',  
  status: CASE  
    WHEN size(users) = 3 THEN 'success'  
    ELSE 'failed'
```

```

END,
created_at: datetime()
})

// Создание связей между логом и админом
CREATE (a)-[:INITIATED_OPERATION]->(l)

RETURN users, l

```

Количество запросов: 1

Задействованные узлы одного типа: 2 (ImportExportLog, User)

Сценарий 10: Экспорт данных

```

MATCH (o:Order)<-[:PLACED_ORDER]-(u:User)
MATCH (o)-[rel:CONTAINS]->(s:Service)
WHERE o.created_at >= datetime('2025-02-01')
      AND o.created_at <= datetime('2025-02-28')
RETURN
  o.order_id AS order_id,
  u.user_id AS user_id,
  u.email AS email,
  s.service_id AS service_id,
  s.service_type AS service_type,
  s.base_price AS base_price,
  r.unit_price AS unit_price,
  o.quantity AS quantity,
  o.status AS status,
  o.created_at AS created_at
ORDER BY o.created_at

MATCH (a:User {user_id: "613dd16f-3d83-4959-b5c4-787c6bd12861"})

```



```

CREATE (l:ImportExportLog {
  log_id: randomUUID(),
  operation_type: 'export',
  status: 'success',
  created_at: datetime()
})
CREATE (a)-[:INITIATED_OPERATION]->(l)
RETURN l

```

Количество запросов: 2

Задействованные узлы одного типа: 4 (Order, User, Service, ImportExportLog)

2.2. Реляционная модель данных

2.2.1 Графическое представление

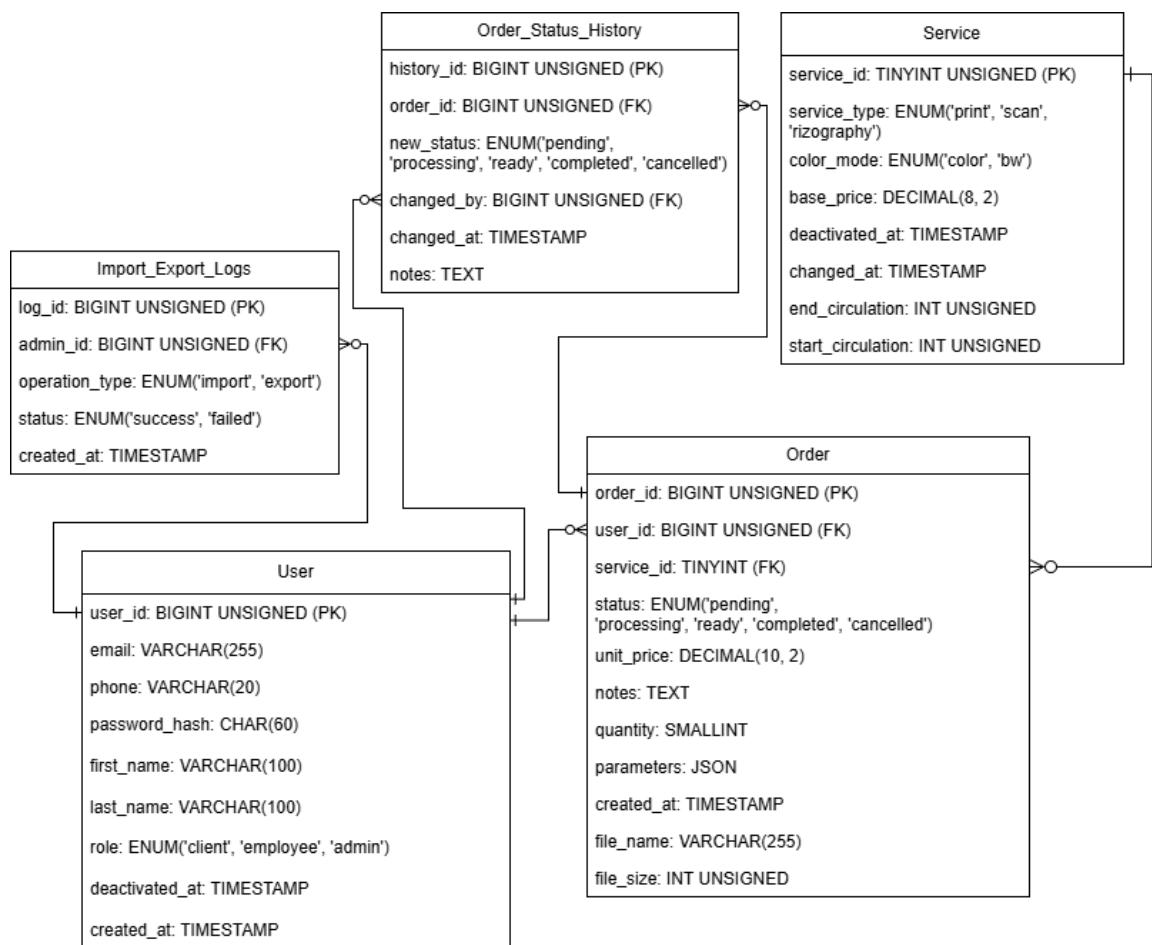


Рисунок 22 — Графическое представление реляционной базы данных

2.2.2 Описание назначений коллекций, типов данных и сущностей

User (Пользователь)

Поле	Тип данных	Размер (байт)	Описание
user_id	BIGINT UNSIGNED	8	Первичный ключ, автоинкремент. Уникальный идентификатор пользователя
email	VARCHAR(255)	257	Уникальный адрес электронной почты, используемый для входа в систему
password_hash	CHAR(60)	60	Хеш пароля (например, bcrypt). Пароли никогда не хранятся в открытом виде
first_name	VARCHAR(100)	102	Имя пользователя
last_name	VARCHAR(100)	102	Фамилия пользователя
phone	VARCHAR(20)	22	Номер телефона (опционально)
role	ENUM('client', 'employee', 'admin')	1	Роль пользователя, определяющая права и доступ к функциям системы
created_at	TIMESTAMP	8	Дата и время создания учетной записи
deactivated_at	TIMESTAMP	8	Дата и время деактивации аккаунта (NULL = аккаунт активен)

Order (Заказ)

Поле	Тип данных	Размер (байт)	Описание
------	------------	------------------	----------

order_id	BIGINT UNSIGNED	8	Первичный ключ, автоинкремент. Уникальный идентификатор заказа
user_id	BIGINT UNSIGNED	8	Внешний ключ, указывающий на клиента, оформившего заказ
service_id	TINYINT UNSIGNED	1	Внешний ключ, указывающий на тип услуги
quantity	SMALLINT UNSIGNED	2	Количество единиц услуги (например, количество листов или страниц)
parameters	JSON	100	Параметры заказа в формате JSON (например, {"format": "A4", "color_mode": "color", "double_sided": true})
unit_price	DECIMAL(10, 2)	12	Цена единицы услуги на момент оформления заказа (историческая, не изменяется при обновлении base_price)
notes	TEXT	100	Комментарий клиента к заказу (опционально)
status	ENUM('pending', 'processing', 'ready', 'cancelled', 'completed')	1	Текущий статус заказа
created_at	TIMESTAMP	8	Дата и время создания заказа
file_name	VARCHAR(255)	257	Оригинальное имя загруженного файла
file_size	INT UNSIGNED	4	Размер файла в байтах

Service (Услуга)

Поле	Тип данных	Размер (байт)	Описание
service_id	TINYINT UNSIGNED	1	Первичный ключ. Уникальный идентификатор услуги
service_type	ENUM('print', 'scan', 'rizography')	1	Тип услуги (например, "Печать")
base_price	DECIMAL(8, 2)	10	Текущая базовая стоимость услуги
color_mode	ENUM('color', 'bw')	1	Тип печати (цветная/чб) (опционально для сканирования/печати)
deactivated_ at	TIMESTAMP	8	Дата и время деактивации услуги (NULL = аккаунт активен)
start_circulat ion	INT UNSIGNED	4	От какого числа тиража актуальна цена (опционально для ризографии)
end_circulati on	INT UNSIGNED	4	До какого числа тиража актуальна цена (опционально для ризографии)
changed_at	TIMESTAMP	8	Дата и время обновления стоимости услуги

Order_Status_History (История статусов заказа)

Поле	Тип данных	Размер (байт)	Описание
history_id	BIGINT UNSIGNED	8	Первичный ключ, автоинкремент

order_id	BIGINT UNSIGNED	8	Внешний ключ на таблицу Order
new_status	ENUM('pending', 'processing', 'ready', 'cancelled', 'completed')	1	Новый статус после изменения
changed_by	BIGINT UNSIGNED	8	Внешний ключ на таблицу Users. ID сотрудника, изменившего статус
changed_at	TIMESTAMP	8	Дата и время изменения статуса
notes	TEXT	200	Описание причины отмены заказа (опционально)

Import_Export_Logs (Журнал импорта/экспорта)

Поле	Тип данных	Размер (байт)	Описание
log_id	BIGINT UNSIGNED	8	Первичный ключ, автоинкремент
admin_id	BIGINT UNSIGNED	8	Внешний ключ на таблицу Users. ID администратора, выполнившего операцию
operation_type	ENUM('import', 'export')	1	Тип операции
status	ENUM('success', 'failed')	1	Статус операции
created_at	TIMESTAMP	8	Дата и время выполнения операции

2.2.3 Оценка удельного объема информации, хранимой в модели

Средний размер одной записи в каждой таблице

User: $8 + 257 + 60 + 102 + 102 + 22 + 1 + 8 + 8 + 24$ (оверхед) = 592 байт

Order: $8 + 8 + 1 + 2 + 100 + 12 + 100 + 1 + 8 + 257 + 4 + 24$ (оверхед) = 525 байт

Service: $1 + 1 + 10 + 1 + 8 + 4 + 4 + 8 + 24$ (оверхед) = 61 байт

Order_Status_History: $8 + 8 + 1 + 8 + 8 + 200 + 24$ (оверхед) = 257 байт

Import_Export_Logs: $8 + 8 + 1 + 1 + 8 + 24$ (оверхед) = 50 байт

*24 байта оверхеда — служебные данные строки (заголовок кортежа, выравнивание)

Коэффициенты зависимости от количества пользователей

Переменная	Значение	Описание
U	переменная	Количество пользователей (базовая переменная)
R_orders	5	Среднее количество заказов на одного пользователя
R_statuschanges	3	Среднее количество изменений статуса на один заказ
D_employees	0.05	Доля сотрудников среди пользователей (5%)
D_admins	0.01	Доля администраторов среди пользователей (1%)
R_impexp	10	Среднее количество операций импорта/экспорта на одного администратора
S_services	6	Количество услуг в справочнике (константа)

Формула общего объёма данных

User: $592 \times U = 592U$

Order: $525 \times (U \times R_orders) = 525 \times 5U = 2625U$

Service: $304 \times S_services = 61 \times 6 = 366$ (константа)

Order_Status_History: $257 \times (U \times R_orders \times R_statuschanges) = 257 \times 5 \times 3 \times U = 3855U$

Import_Export_Logs: $50 \times (U \times D_admins \times R_impexp) = 50 \times 0.01 \times 10 \times U = 5U$

Итоговая формула зависимости объёма от количества пользователей

$$V_{\text{actual}}(U) = 592U + 2625U + 366 + 3855U + 5U$$

$$V_{\text{actual}}(U) = 7077 \times U + 366$$

Избыточность данных

Избыточность модели данных это отношение между фактическим объемом модели и «чистым» объемом данных.

Фактический объем модели: $V_{\text{actual}}(U) = 7077U + 366$

«Чистый» объем данных:

User (email + first_name + last_name + password_hash + phone + role): $257 + 102 + 102 + 60 + 22 + 1 = 544 \Rightarrow 544U$

Order (quantity + parameters + unit_price + note + status + file_name + file_size): $2 + 100 + 12 + 100 + 1 + 257 + 4 = 476 \Rightarrow 476 \times 5U = 2380U$

Service (service_name + base_price + description + service_type): $1 + 10 + 4 + 4 = 19 \Rightarrow 19 \times 6 = 114$ (константа)

Order_Status_History (new_status + comment): $1 + 200 = 201 \Rightarrow 201 \times 15U = 3015U$

Import_Export_Logs (operation_type + status): $1 + 1 = 2 \Rightarrow 2 \times 0.1U = 0.2U$

$$V_{\text{clean}}(U) = 544U + 2380U + 114 + 3015U + 0.2U$$

$$V_{\text{clean}}(U) = 5939.2U + 114$$

Формула избыточности

$$\text{Redundancy}(U) = V_{\text{actual}}(U) / V_{\text{clean}}(U) = (7077 \times U + 366) / (5939.2 \times U + 114)$$

2.2.4 Запросы к модели, с помощью которых реализуются сценарии использования

Сценарий 1: Регистрация на сайте

```
-- Проверка существования пользователя с таким email
SELECT user_id FROM User
WHERE email = 'newuser@mail.ru'
LIMIT 1;
```

-- Вставка нового пользователя

```
INSERT INTO User (email, password_hash, first_name, last_name, phone, role,  
created_at, deactivated_at)  
VALUES ('newuser@mail.ru', '$2y$10$xyz789...', 'Иван', 'Иванов', '+79001234567',  
'client', NOW(), NULL);
```

-- Получение данных созданного пользователя для сессии

```
SELECT user_id, email, first_name, last_name, role, created_at  
FROM User  
WHERE email = 'newuser@mail.ru' AND deactivated_at IS NULL;
```

Количество запросов: 3

Задействованные таблицы: 1 (User)

Сценарий 2: Авторизация на сайте

-- Проверка учетных данных

```
SELECT user_id, password_hash, role, first_name, last_name  
FROM User  
WHERE email = 'ivanov@mail.ru' AND deactivated_at IS NULL;
```

Количество запросов: 1

Задействованные таблицы: 1 (User)

Сценарий 3: Оформление заказа полиграфической услуги

-- Получение базовой стоимости услуги (для валидации/отображения клиенту)

```
SELECT base_price FROM Service  
WHERE service_id = 1 AND deactivated_at IS NULL;
```

-- Вставка нового заказа с фиксацией цены на момент оформления

```
INSERT INTO `Order` (  
user_id, service_id, unit_price, quantity, parameters,
```



```

    notes, status, created_at, file_name, file_size
) VALUES (
    1, 1, 20.00, 50, '{"format": "A4", "color": "bw"}',
    'Срочно, к 15:00', 'pending', NOW(), 'document.pdf', 2048576
);

-- Получение данных созданного заказа
SELECT LAST_INSERT_ID() AS order_id;

-- Создание записи в истории статусов
INSERT INTO Order_Status_History (order_id, new_status, changed_by,
changed_at, notes)
VALUES (LAST_INSERT_ID(), 'pending', 1, NOW(), NULL);

-- Получение полной информации о заказе
SELECT
    o.order_id, o.service_id, s.service_type, s.color_mode,
    o.unit_price, o.quantity,
    o.quantity * o.unit_price AS total_price, -- расчёт итоговой стоимости
    o.parameters, o.notes, o.status,
    o.created_at, o.file_name, o.file_size
FROM `Order` o
JOIN Service s ON o.service_id = s.service_id
WHERE o.order_id = LAST_INSERT_ID();

```

Количество запросов: 5

Задействованные таблицы: 3 (Order, Service, Order_Status_History)

Сценарий 4: Просмотр детальной информации о заказе

```

-- Получение детальной информации по конкретному заказу
SELECT

```

```

    o.order_id, o.service_id, s.service_type, s.color_mode,
    o.unit_price, o.quantity,
    o.quantity * o.unit_price AS total_price, -- расчёт суммы
    o.parameters, o.notes, o.status,
    o.created_at, o.file_name, o.file_size
FROM `Order` o
JOIN Service s ON o.service_id = s.service_id
WHERE o.order_id = 101 AND o.user_id = 1;

-- Получение истории статусов заказа
SELECT
    h.new_status, h.changed_at, h.notes,
    u.first_name, u.last_name
FROM Order_Status_History h
JOIN User u ON h.changed_by = u.user_id
WHERE h.order_id = 101
ORDER BY h.changed_at ASC;

```

Количество запросов: 2

Задействованные таблицы: 4 (Order, Service, Order_Status_History, User)

Сценарий 5: Обработка заказа

```

-- Получение списка заказов в очереди
SELECT
    o.order_id, o.user_id, u.first_name, u.last_name, u.phone,
    o.service_id, s.service_type, s.color_mode,
    o.unit_price, o.quantity,
    o.quantity * o.unit_price AS total_price,
    o.parameters, o.file_name, o.file_size,
    o.status, o.created_at
FROM `Order` o

```

```

JOIN User u ON o.user_id = u.user_id
JOIN Service s ON o.service_id = s.service_id
WHERE o.status = 'pending'
ORDER BY o.created_at ASC;

-- Обновление статуса заказа
UPDATE `Order`
SET status = 'completed'
WHERE order_id = 101;

-- Вставка записи в историю статусов
INSERT INTO Order_Status_History (order_id, new_status, changed_by,
changed_at, notes)
VALUES (101, 'completed', 2, NOW(), NULL);

```

Количество запросов: 3

Задействованные таблицы: 4 (Order, User, Service, Order_Status_History)

Сценарий 6: Редактирование стоимости услуги

```

-- Получение списка всех услуг
SELECT service_id, service_type, base_price, color_mode,
       start_circulation, end_circulation, deactivated_at, changed_at
FROM Service
WHERE deactivated_at IS NULL
ORDER BY service_type ASC, service_id ASC;

-- Обновление стоимости услуги
UPDATE Service
SET base_price = 6.00, changed_at = NOW()
WHERE service_id = 1;

```

-- Подтверждение обновления

```
SELECT service_id, service_type, base_price, changed_at  
FROM Service WHERE service_id = 1;
```

Количество запросов: 3

Задействованные таблицы: 1 (Service)

Сценарий 7: Выдача роли для нового сотрудника

-- Получение списка всех пользователей с ролями

```
SELECT user_id, first_name, last_name, email, role, created_at  
FROM User  
WHERE deactivated_at IS NULL  
ORDER BY created_at DESC;
```

-- Обновление роли пользователя

```
UPDATE User  
SET role = 'employee'  
WHERE user_id = 4;
```

-- Подтверждение обновления

```
SELECT user_id, first_name, last_name, role FROM User WHERE user_id = 4;
```

Количество запросов: 3

Задействованные таблицы: 1 (User)

Сценарий 8: Построение кастомизированной статистики

-- Статистика по услугам за период

```
SELECT  
    s.service_type,  
    COUNT(o.order_id) AS orders_count,  
    SUM(o.quantity * o.unit_price) AS total_revenue -- ← используем unit_price из  
Order
```

```

FROM `Order` o
JOIN Service s ON o.service_id = s.service_id
WHERE o.created_at BETWEEN '2025-02-01' AND '2025-02-28'
    AND o.status IN ('completed', 'ready')
GROUP BY s.service_type
ORDER BY total_revenue DESC;

-- Статистика по сотрудникам за период
SELECT
    u.first_name, u.last_name,
    COUNT(DISTINCT h.order_id) AS orders_processed,
    SUM(o.quantity * o.unit_price) AS total_revenue -- ← используем unit_price из
Order
FROM Order_Status_History h
JOIN User u ON h.changed_by = u.user_id
JOIN `Order` o ON h.order_id = o.order_id
JOIN Service s ON o.service_id = s.service_id
WHERE h.changed_at BETWEEN '2025-02-01' AND '2025-02-28'
    AND u.role = 'employee'
GROUP BY u.user_id, u.first_name, u.last_name
ORDER BY orders_processed DESC;

```

Количество запросов: 2

Задействованные таблицы: 4 (Order, Service, User, Order_Status_History)

Сценарий 9: Импорт данных

```

-- Начало транзакции
START TRANSACTION;

-- Вставка импортируемых пользователей (пакетно, пример для 3
пользователей)

```

```

INSERT INTO User (email, password_hash, first_name, last_name, role, created_at,
deactivated_at)
VALUES
    ('import1@mail.ru', '$2y$10$...', 'Импорт', 'Первый', 'client', NOW(), NULL),
    ('import2@mail.ru', '$2y$10$...', 'Импорт', 'Второй', 'client', NOW(), NULL),
    ('import3@mail.ru', '$2y$10$...', 'Импорт', 'Третий', 'client', NOW(), NULL);

-- Логирование операции импорта
INSERT INTO Import_Export_Logs (admin_id, operation_type, status, created_at)
VALUES (3, 'import', 'success', NOW());

-- Фиксация транзакции
COMMIT;

```

Количество запросов: 4

Задействованные таблицы: 2 (User, Import_Export_Logs)

Сценарий 10: Экспорт данных

```

-- Экспорт всех заказов за период
SELECT o.order_id, o.user_id, u.email,
    s.service_id, s.service_type, s.base_price, s.color_mode,
    o.quantity, o.parameters, o.status, o.created_at
FROM `Order` o
JOIN User u ON o.user_id = u.user_id
JOIN Service s ON o.service_id = s.service_id
WHERE o.created_at BETWEEN '2025-02-01' AND '2025-02-28';

-- Логирование операции экспорта
INSERT INTO Import_Export_Logs (admin_id, operation_type, status, created_at)
VALUES (3, 'export', 'success', NOW());

```

Количество запросов: 2

Задействованные таблицы: 4 (Order, User, Service, Import_Export_Logs)

2.3. Сравнение моделей

2.3.1 Удельный объем информации

$$V_{\text{actual}}^{\text{neo4j}}(U) = 16132,2U + 2422$$

$$V_{\text{actual}}^{\text{sql}}(U) = 7077U + 366$$

$$V_{\text{actual}}^{\text{neo4j}}(U) / V_{\text{actual}}^{\text{sql}}(U) = (16132,2U + 2422) / (7077U + 366) \approx 2,29$$

Графовая модель требует примерно в 2,29 раза больше памяти, чем реляционная, из-за хранения связей как отдельных сущностей и наличия служебных данных.

2.3.2 Запросы по отдельным юзкейсам

Сценарий	Neo4j (запросы)	SQL (запросы)	Узлы / таблицы
Регистрация	2	3	1
Авторизация	1	1	1
Оформление заказа	2	5	3
Просмотр заказа	2	2	4
Обработка заказа	2	3	4
Редактирование услуги	2	3	1
Назначение роли	2	3	1
Статистика	2	2	4
Импорт	1	4	2
Экспорт	2	2	4

Среднее количество запросов:

Neo4j: 1-2

Sql: 2-5

Графовая модель требует меньше запросов за счет возможности выполнять навигацию по связям внутри одного запроса.

2.3.3 Вывод

Таким образом, графовая модель Neo4j требует большего объема памяти (примерно в 2,3 раза), однако обеспечивает быстрое выполнение операций, связанных с обходом связей между сущностями. В сценариях с высокой связанностью данных (заказы, история статусов, аналитика) Neo4j выполняет меньше запросов и избегает дорогостоящих операций JOIN, что делает ее более подходящей для таких задач. Реляционная модель более эффективна по памяти и лучше подходит для простых CRUD-операций и структурированных данных.

В условиях предметной области - полиграфического центра - графовая модель позволяет сократить количество запросов, затрагивающих несколько связанных сущностей, и упростить их структуру, поэтому целесообразнее использовать модель neo4j, несмотря на больший объем хранения, поскольку она обеспечивает более эффективную обработку взаимосвязанных данных.

3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

3.1. Описание приложения и использованные технологии

Разработанное веб-приложение представляет собой информационную систему для автоматизации работы центра полиграфических услуг. Приложение построено на трехуровневой архитектуре, включающей уровень представления (frontend), уровень бизнес-логики (backend) и уровень хранения данных (база данных).

Бэкенд построен по принципу слоёной архитектуры:

- Маршруты (routes) — определяют эндпоинты API и связывают их с контроллерами.
- Контроллеры (controllers) — обрабатывают запросы, вызывают соответствующие модели и формируют ответы (databaseController.js, orderController.js, serviceController.js, userController.js).
- Middleware — выполняют вспомогательные функции: проверка JWT-токена и авторизация (authMiddleware.js), валидация входных данных (orderValidator.js, serviceValidator.js, userValidator.js), обработка загрузки файлов (uploadFileMiddleware.js).
- Модели (models) — реализуют слой доступа к базе данных Neo4j. Каждая модель (databaseModel.js, orderModel.js, serviceModel.js, userModel.js) содержит методы для выполнения соответствующих Cypher-запросов к графовой базе данных.
- Сервисы (services) — содержат бизнес-логику (расчёт стоимости, проверка ролей, управление статусами).

Фронтенд представляет собой одностраничное приложение (SPA) на Vue, взаимодействующее с сервером через REST API.

Использованные технологии:

Фронтенд:

- Vue.js — создание пользовательского интерфейса (SPA)
- Vue Router — маршрутизация между страницами
- Axios — выполнение HTTP-запросов к серверу

- Pinia — управление состоянием приложения
- Naive UI — библиотека компонентов пользовательского интерфейса
- pdf-lib — работа с PDF-документами
- HTML5, CSS3 — вёрстка и стилизация интерфейса

Бэкенд:

- Node.js — среда выполнения JavaScript на сервере
- Express.js — создание REST API
- jsonwebtoken (JWT) — аутентификация и авторизация пользователей
- bcrypt — хеширование паролей
- multer — обработка загрузки файлов

База данных:

- Neo4j — графовое хранение данных
- neo4j-driver — официальный JavaScript-драйвер для взаимодействия с Neo4j

Инструменты:

- Git — контроль версий
- Docker — контейнеризация приложения и базы данных
- npm — менеджер пакетов

3.2. Снимки экрана приложения

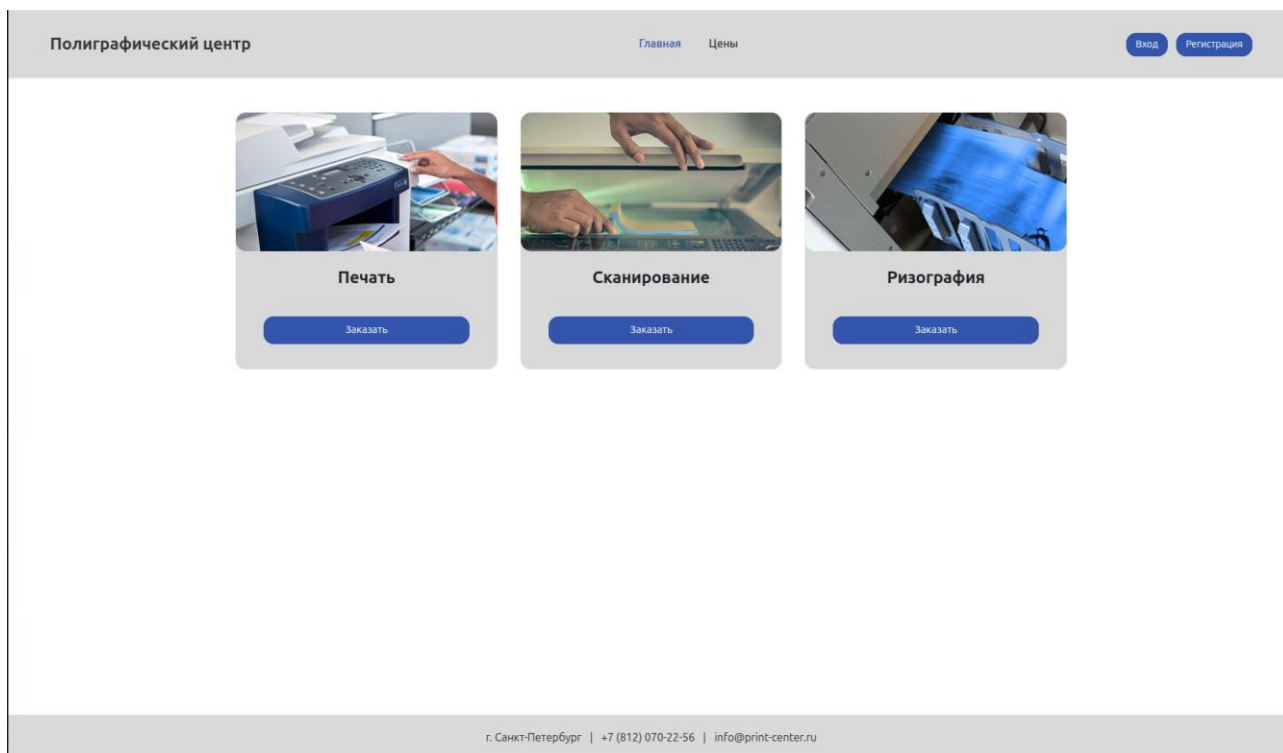


Рисунок 23 — Главная страница для неавторизованного пользователя



Рисунок 24 — Вкладка «Цены» для неавторизованного пользователя

Полиграфический центр

ГлавнаяЦены

ВходРегистрация

Регистрация

Фамилия *

Введите фамилию

Имя *

Введите имя

Номер телефона *

+7 (999) 123-45-67

Email *

example@mail.ru

Пароль *

Введите пароль

Подтверждение пароля *

Повторите пароль

Отмена

Зарегистрироваться

Уже есть аккаунт? Войти

г. Санкт-Петербург | +7 (812) 070-22-56 | info@print-center.ru

Рисунок 25 — Форма регистрации

Полиграфический центр

ГлавнаяЦены

ВходРегистрация

Вход

Email *

example@mail.ru

Пароль *

Введите пароль

Войти

Нет аккаунта? Зарегистрироваться

г. Санкт-Петербург | +7 (812) 070-22-56 | info@print-center.ru

Рисунок 26 — Форма логина

Полиграфический центр

Главная

Цены

Кабинет

Выйти

Оформление заказа

Количество копий

2

— +

Формат

A4

▼

Режим цвета

☒ Цветное

☐ Черно-белое

Файл для печати

Выбрать файл

Пояснительная записка.pdf (1017.3 KB) X

✓

Стоимость: 720 руб.

Расчёт: 15 Р (базовая цена) × 24 стр. × 2 шт. = 720 Р

Комментарий к заказу

Дополнительные пожелания к заказу...

0 / 500

Рассчитать

Оформить заказ

г. Санкт-Петербург | +7 (812) 070-22-56 | info@print-center.ru

Рисунок 27 — Оформление заказа на печать

Полиграфический центр

Главная

Цены

Кабинет

Выйти

Заказ №735F3651

Дата оформления:

17.05.2026, 16:39:56

Статус:

В обработке

Итого:

720 руб

Тип: Печать

Файл: [order-1779025196326-21193056.pdf](#)

Формат: A4

Количество страниц: 24

Комментарий: —

Цветность: Цветная

Количество копий: 2

г. Санкт-Петербург | +7 (812) 070-22-56 | info@print-center.ru

Рисунок 28 — Страница заказа для клиента

Полиграфический центр

Главная Цены

Кабинет Выйти

Оформление заказа

Количество копий

2 — +

Формат

A4

Режим цвета

☒ Цветное ☐ Черно-белое

Качество скана

☐ Сохранить качество ☒ Сжать файл

Время посещения

15:00 – 17:00

✓ **Стоимость: 20 руб.**
Расчёт: 10 Р (базовая цена) × 2 шт. = 20 Р

Комментарий к заказу

Дополнительные пожелания к заказу...

0 / 500

Рассчитать

Оформить заказ

Рисунок 29 — Оформление заказа на сканирование

Полиграфический центр

Главная Цены

Кабинет Выйти

Добро пожаловать, Client!

Личные данные

Мои заказы

Личные данные

Имя	Фамилия
Client	One
Телефон	Email
+79990003331	client1@print.local
Создан:	Id:
10.05.2026, 12:27:01	4b7fc500-bc46-4242-8828-8386337e6556

г. Санкт-Петербург | +7 (812) 070-22-56 | info@print-center.ru

Рисунок 30 — Личный кабинет с открытой вкладкой «Личные данные»

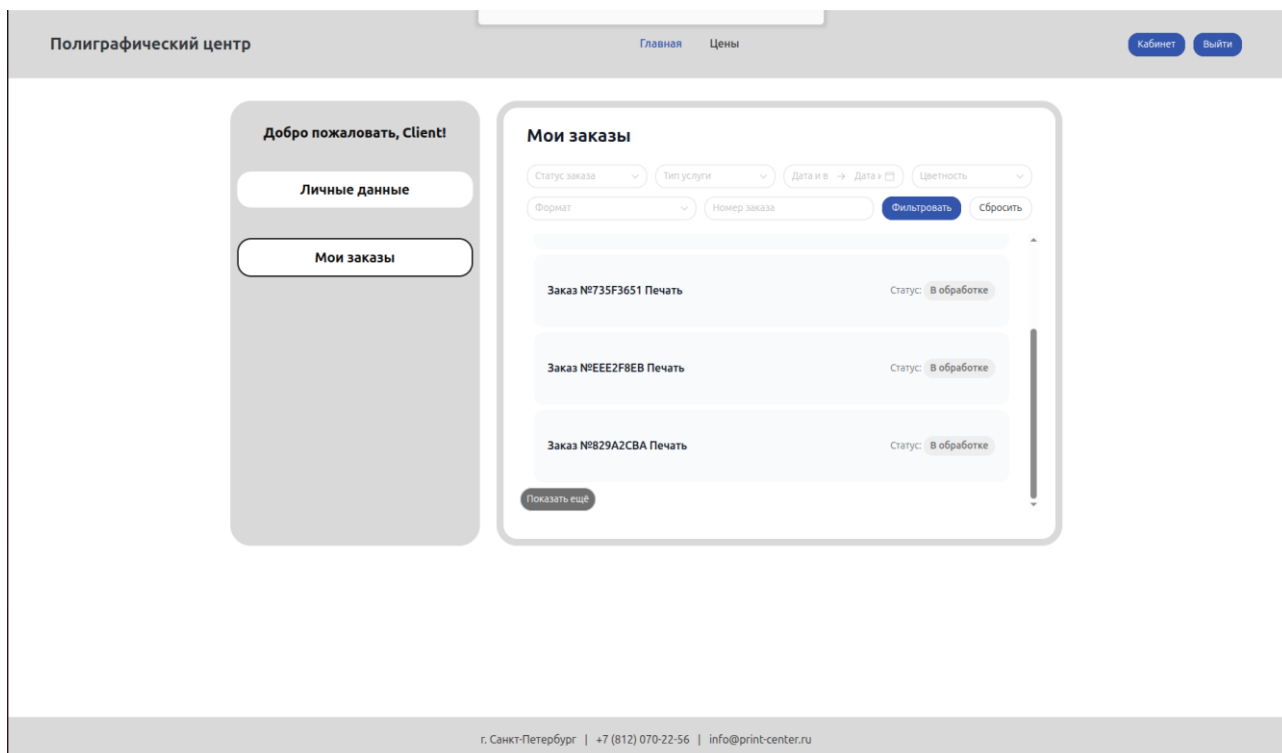


Рисунок 31 - Личный кабинет с открытой вкладкой «Мои заказы»

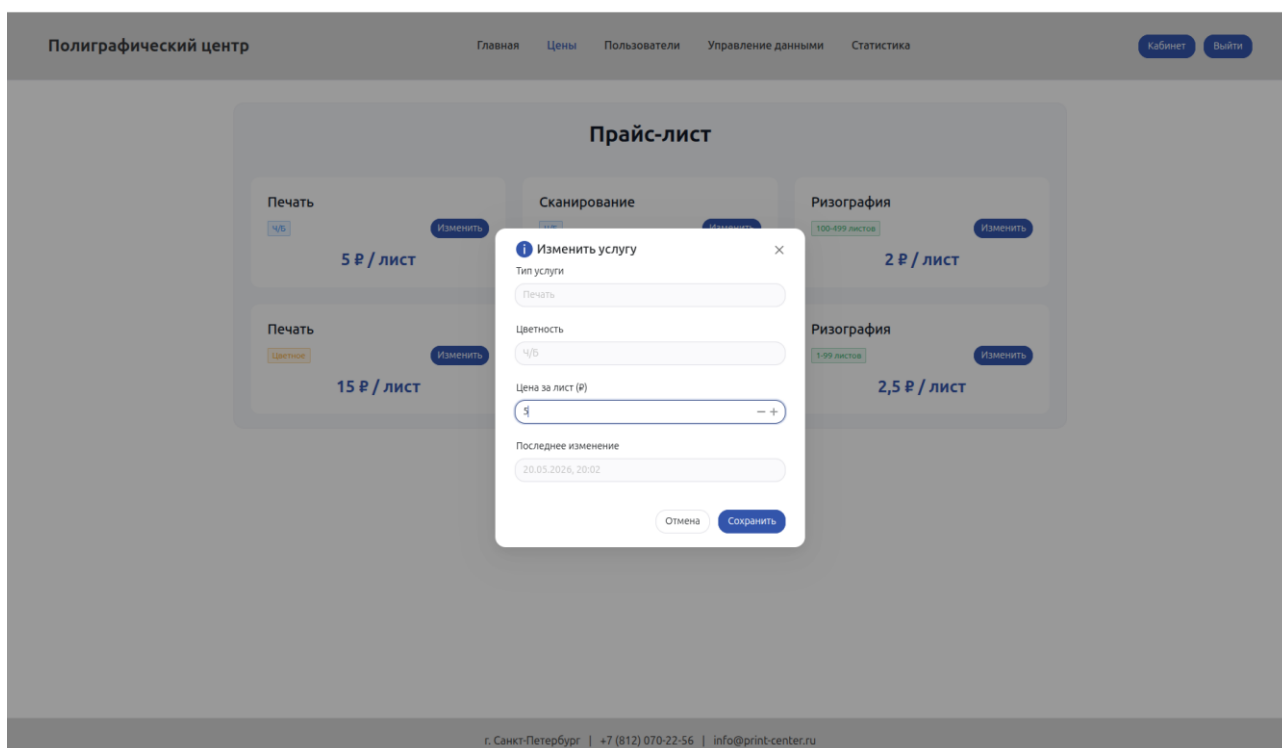


Рисунок 32 — Редактирование стоимости услуги администратором

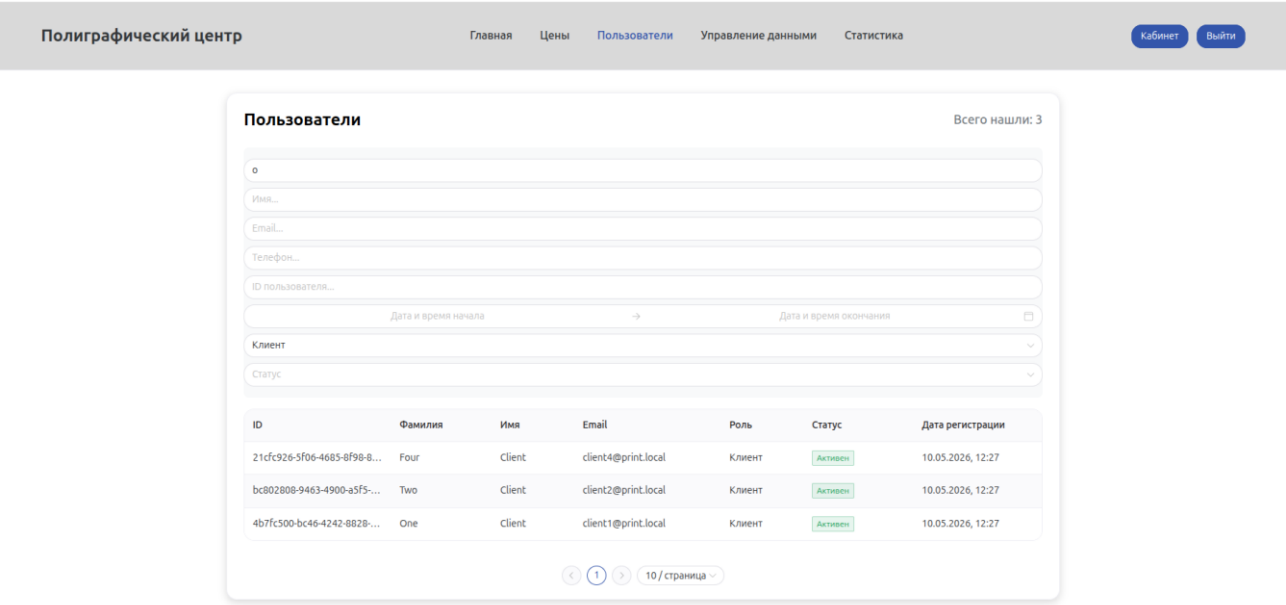


Рисунок 33 — Управление пользователями для администратора

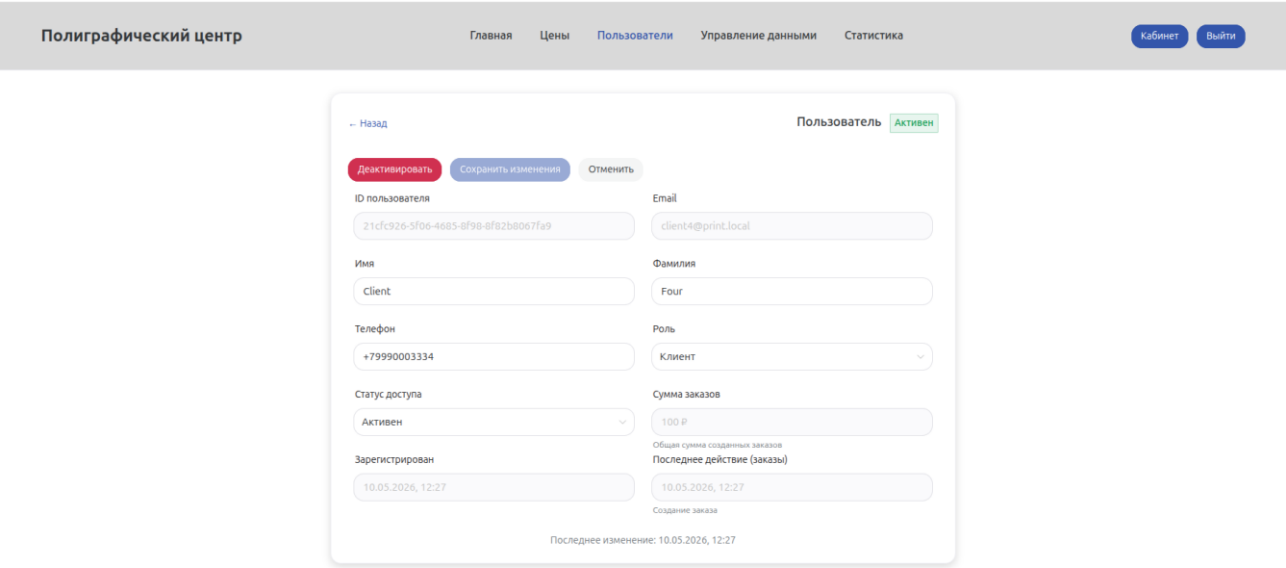


Рисунок 34 — Страница определенного пользователя для администратора

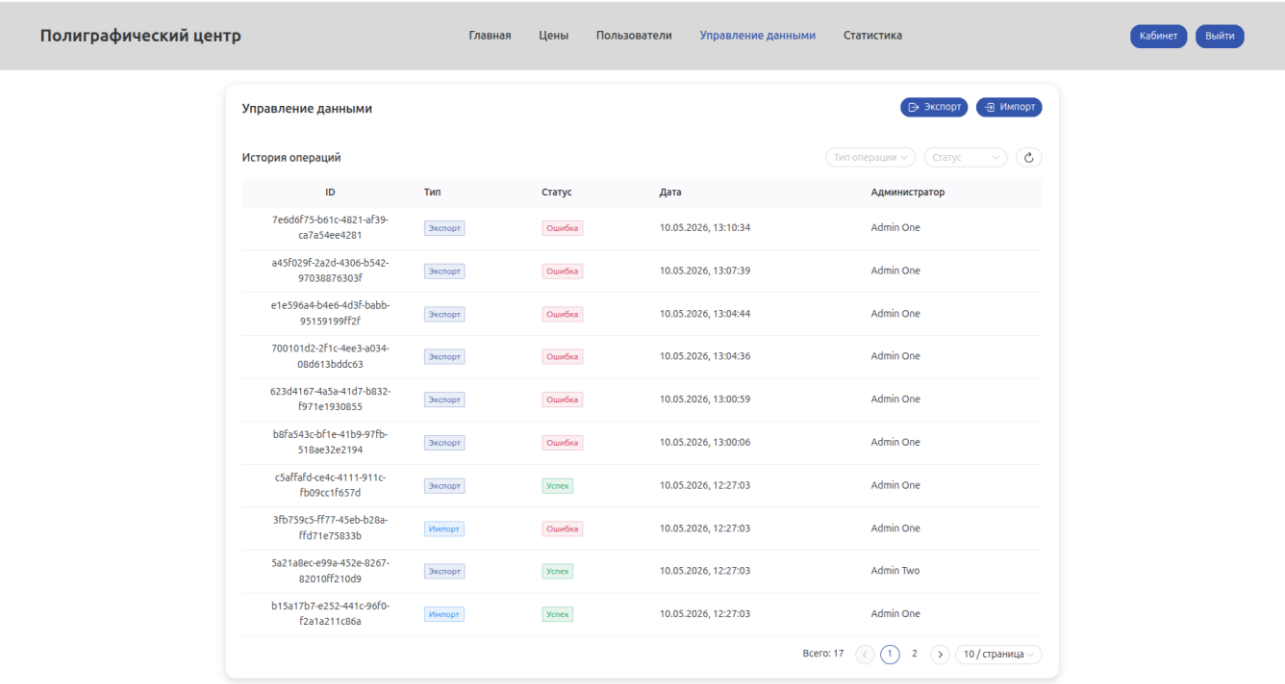


Рисунок 35 — Управление данными для администратора

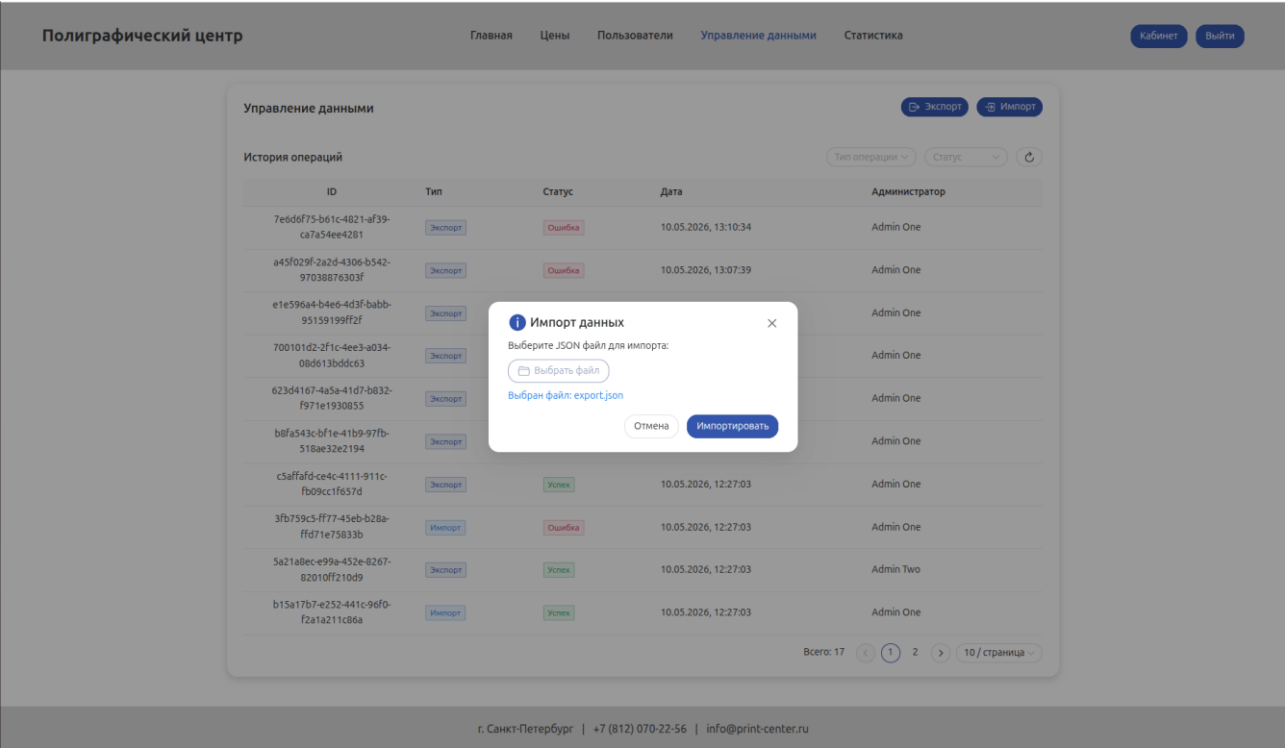


Рисунок 36 — Импорт данных для администратора

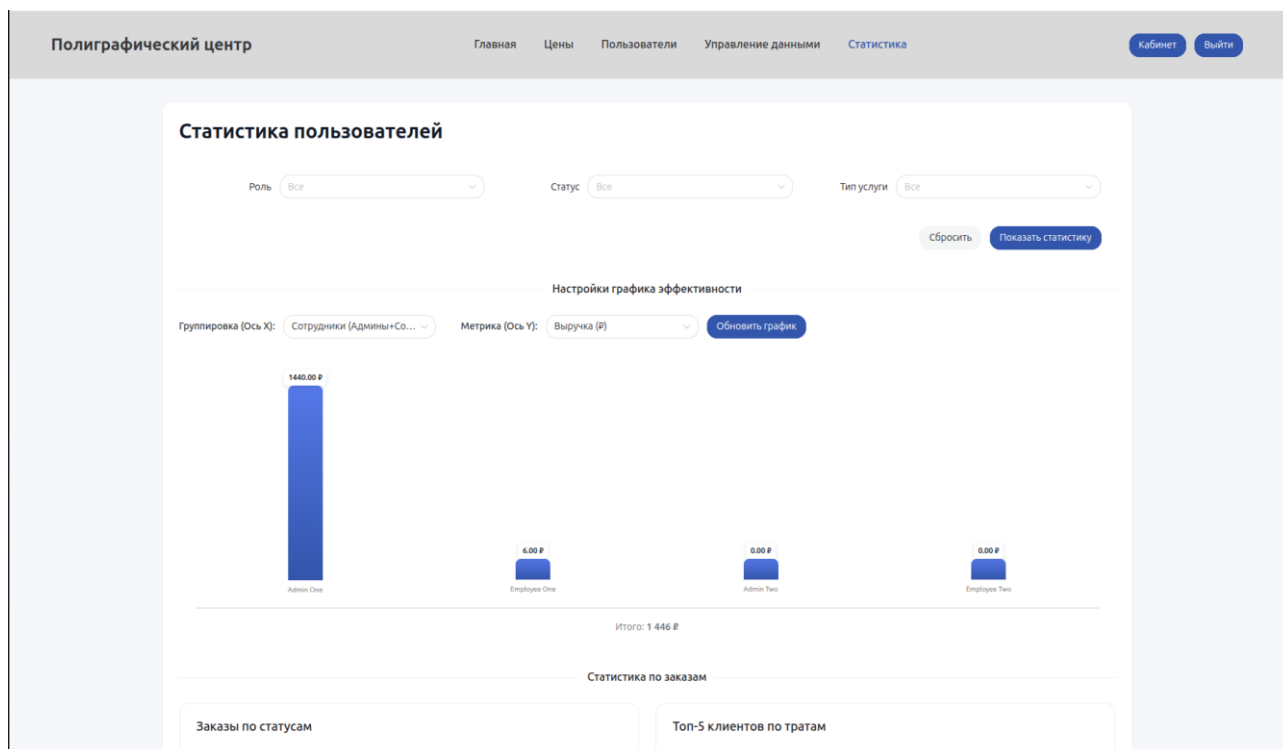


Рисунок 37 — Построение кастомизируемой статистики администратором

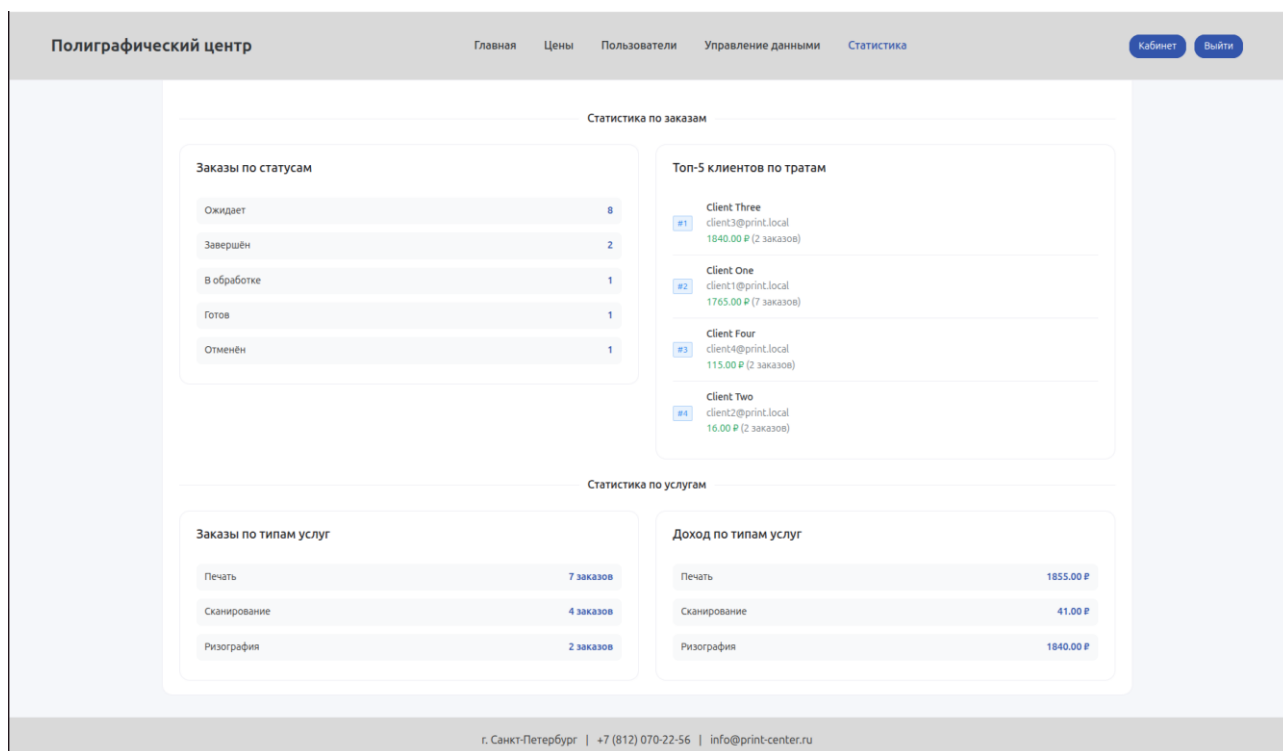


Рисунок 38 — Общая статистика для администратора

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

В ходе выполнения работы было разработано веб-приложение — информационная система для автоматизации работы центра полиграфических услуг. В результате были достигнуты следующие результаты:

- Реализован полный функционал согласно техническому заданию:
 - Клиенты могут оформлять заказы на печать, сканирование и ризографию, просматривать статусы и историю заказов.
 - Сотрудники могут принимать и выполнять заказы, изменять их статусы, вести клиентскую базу.
 - Администраторы могут управлять пользователями, услугами и тарифами, выполнять импорт и экспорт данных, строить кастомизированную статистику.
- Разработана трехуровневая архитектура с чётким разделением на фронтенд (Vue.js), бэкенд (Node.js + Express) и базу данных (Neo4j), что обеспечивает масштабируемость и удобство сопровождения.

Недостатки и пути для улучшения полученного решения

Несмотря на достигнутые результаты, разработанное решение имеет следующие недостатки и пути их улучшения:

1. Отсутствие уведомлений для клиентов
Путь улучшения: реализовать отправку email- или SMS-уведомлений при изменении статуса заказа.
2. Нет возможности массовой печати файлов
Путь улучшения: добавить функцию пакетной загрузки и печати нескольких файлов одним заказом.
3. Интерфейс не адаптирован для мобильных устройств
Путь улучшения: добавить адаптивную вёрстку или разработать мобильное приложение.

Будущее развитие решения

Дальнейшее развитие информационной системы может включать следующие направления:

1. Исправление недостатков указанных выше.
2. Система лояльности — накопительные бонусы, скидки для постоянных клиентов, персональные предложения.
3. Добавление новых типов услуг — ламинация, брошюровка, резка, тиснение и другие полиграфические услуги.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация neo4j // Neo4j documentation URL: <https://neo4j.com/docs/>
2. Наш проект. URL: <https://github.com/moevm/nsql1h26-print>

ПРИЛОЖЕНИЕ А

ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ

1. Склонировать исходный код из репозитория (см. ссылку в списке используемой литературы).
2. При необходимости изменить значение переменных окружения в файле `.env`
3. Запуск осуществляется по команде `docker-compose up -d --build`, после успешного запуска фронтенд должен быть доступен по порту 8080, бекенд – 3000.
4. Завершение работы производится по команде `docker-compose down`.

ПРИЛОЖЕНИЕ В

ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

1. Регистрация и вход в систему

1. Регистрация:

- Перейдите на главную страницу приложения.
- Нажмите кнопку «Регистрация».
- Заполните поля: email, имя, фамилия, номер телефона, пароль.
- Нажмите «Зарегистрироваться».
- После успешной регистрации вы будете автоматически авторизованы.

2. Вход в систему:

- Нажмите кнопку «Вход».
- Введите email и пароль.
- Нажмите «Войти».
- В зависимости от вашей роли (клиент, сотрудник, администратор) откроется соответствующий интерфейс.

2. Работа в роли «Клиент»

1. Создание нового заказа:

- На главной странице выберете услугу.
- Загрузите файл (для печати или ризографии).
- Укажите параметры: количество копий, цветность, формат и т.д.
- Нажмите «Рассчитать стоимость» — система покажет итоговую цену.
- Подтвердите заказ кнопкой «Оформить».

2. Просмотр статуса заказа:

- Перейдите в «Личный кабинет» и выберите «Мои заказы».
- Вы увидите список всех своих заказов с текущими статусами:
 - «Ожидает» — заказ принят, ожидает обработки;
 - «В очереди» — сотрудник одобрил заказ и он находится в очереди заказов;

- «Готов к выдаче» — заказ выполнен;
- «Отменён» — заказ отменён.
- «Завершен» - заказ получен клиентом и завершен.

3. Работа в роли «Сотрудник»

1. Просмотр всех заказов:

- На главной странице отображается список всех активных заказов.
- Для фильтрации используйте фильтры вверху страницы.

2. Приём и выполнение заказа:

- Выберите заказ из списка.
- Нажмите «Принять», если заказ возможно выполнить — статус изменится на «В очереди». Если же заказ некорректен, нажмите «Отменить» и опишите причину отмены.
- После выполнения заказа нажмите «Готов к выдаче» — статус изменится на «Готов к выдаче».

4. Работа в роли «Администратор»

1. Управление пользователями:

- Перейдите в раздел «Пользователи».
- Вы увидите список всех пользователей с их ролями.
- Для редактирования нажмите на пользователя — можно изменить роль, личные данные, номер телефона или заблокировать/разблокировать учётную запись.

2. Управление тарифами:

- Перейдите в раздел «Цены».
- Вы увидите список доступных услуг с ценами.
- Для редактирования существующей услуги нажмите на кнопку «Изменить» внутри нужной услуги.

3. Импорт и экспорт данных:

- Перейдите в раздел «Управление данными».
- Для экспорта: нажмите «Экспорт».

- Для импорта: нажмите «Импорт», выберите файл и нажмите «Импортировать».
4. Построение кастомизированной статистики:
- Перейдите в раздел «Статистика».
 - Выберите параметры для осей.
 - После выбора параметров автоматически появится диаграмма.